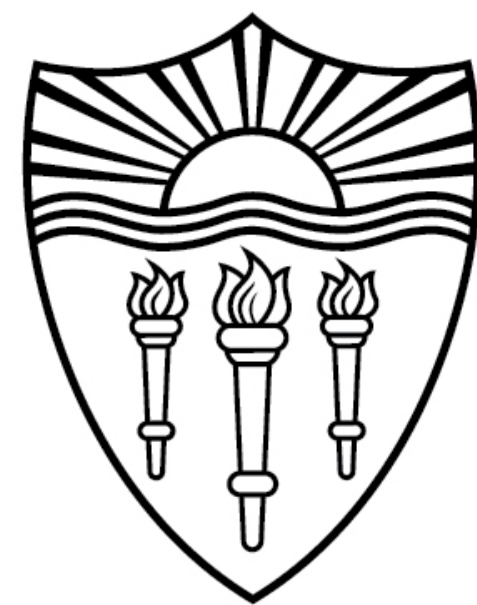


CSCI 544: Applied Natural Language Processing

Recurrent Neural Networks for Sequence Labeling

Xuezhe Ma (Max)

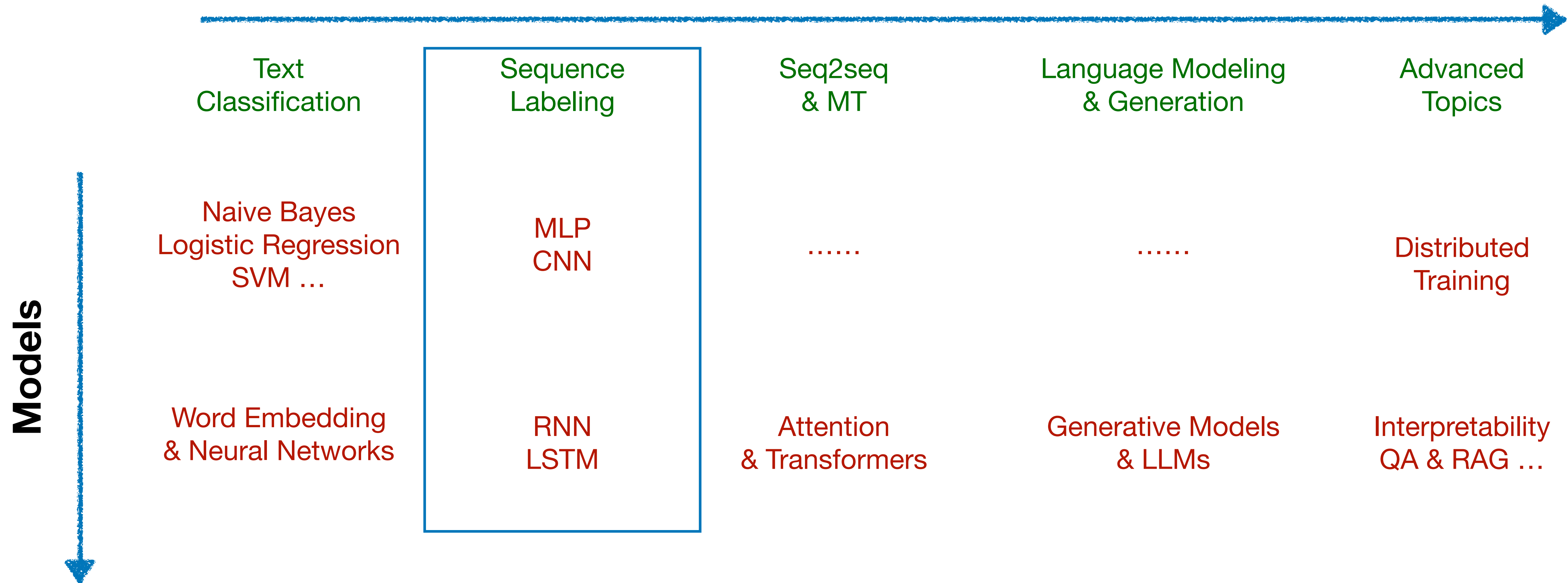


USC University of
Southern California

Quiz Passkey: 27Jan544

Course Organization

NLP Tasks



Structured Prediction & Sequence Labeling

What is Structured Prediction

- Unstructured Prediction
 - Output Y consists of a **single component**



X



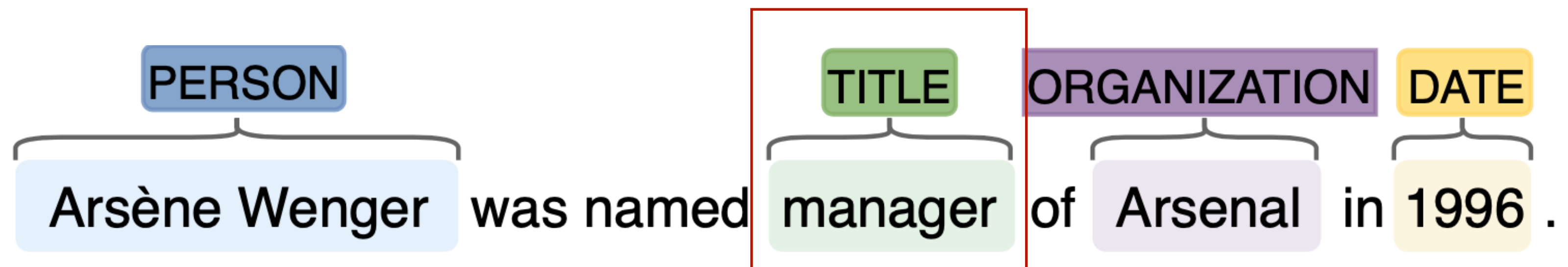
{**Cat**, Dog, Finch, Owl}

Y

Structured Prediction

- Y consists of **multiple components** $Y = \{y_1, y_2, \dots, y_n\}$

Named Entity Recognition



$$y_i = \langle \mathbf{manager} \rightarrow \mathbf{Title} \rangle$$

Structured Prediction

- Y consists of **multiple components** $Y = \{y_1, y_2, \dots, y_n\}$

麦克斯 在 南加大 工作

X



Max is working at USC

↑
 Y_1

↑
 Y_2

↑
 Y_3

↑
 Y_4

↑
 Y_5

Structured Prediction

- Y consists of **multiple components** $Y = \{y_1, y_2, \dots, y_n\}$
- **(Strong) correlations** between output components

麦克斯 在 南加大 工作

X



Max is working at USC

↑
 Y_1

↑
 Y_2

↑
 Y_3

↑
 Y_4

↑
 Y_5

Structured Prediction

- Y consists of **multiple components** $Y = \{y_1, y_2, \dots, y_n\}$
- **(Strong) correlations** between output components
- **Exponential** output space
 - Decoding: $y^* = \operatorname{argmax}_{y \in \mathcal{Y}} p(y | x)$

麦克斯 在 南加大 工作

X



Max is working at USC

$\uparrow$
 Y_1

$\uparrow$
 Y_2

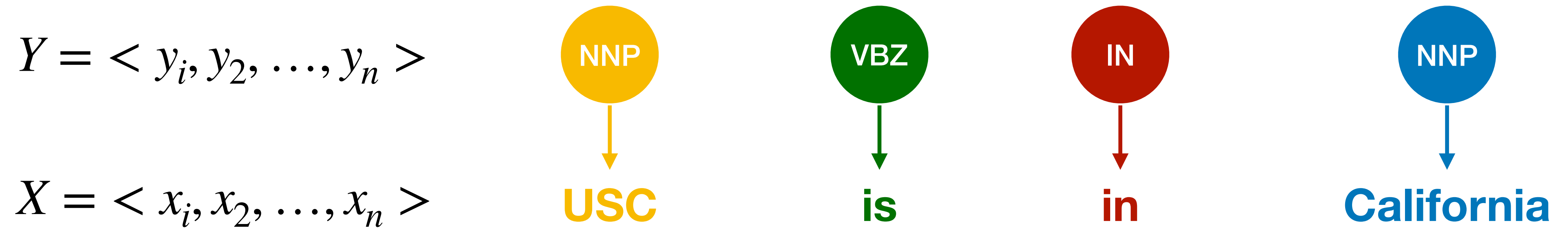
$\uparrow$
 Y_3

$\uparrow$
 Y_4

$\uparrow$
 Y_5

What is Sequence Labeling?

A type of structured prediction tasks



Assigning each token of X , e.g. x_i a corresponding label y_i

Why Sequence Labeling?

Part-of-Speech Tagging

PRP VBD DT NN IN DT NN
I saw a girl with a telescope

Named Entity Recognition

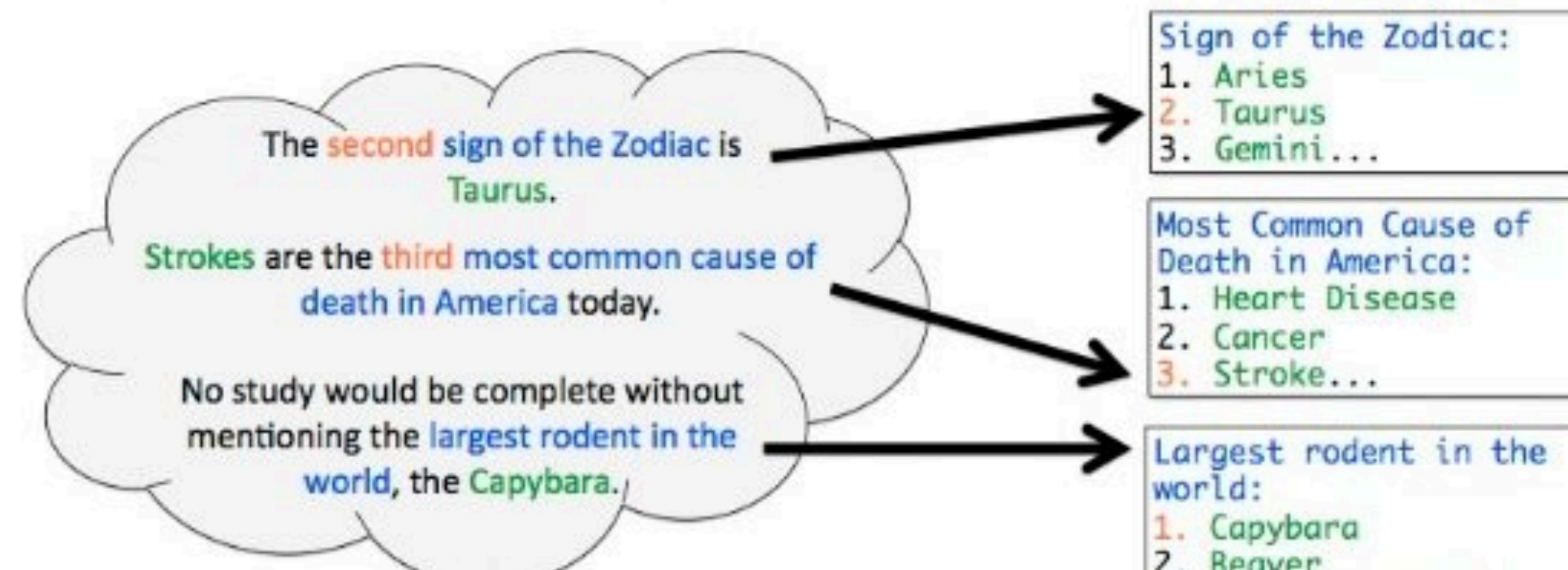
PERSON TITLE ORGANIZATION DATE
Arsène Wenger was named manager of Arsenal in 1996 .

Information Extraction

Unstructured Web Text

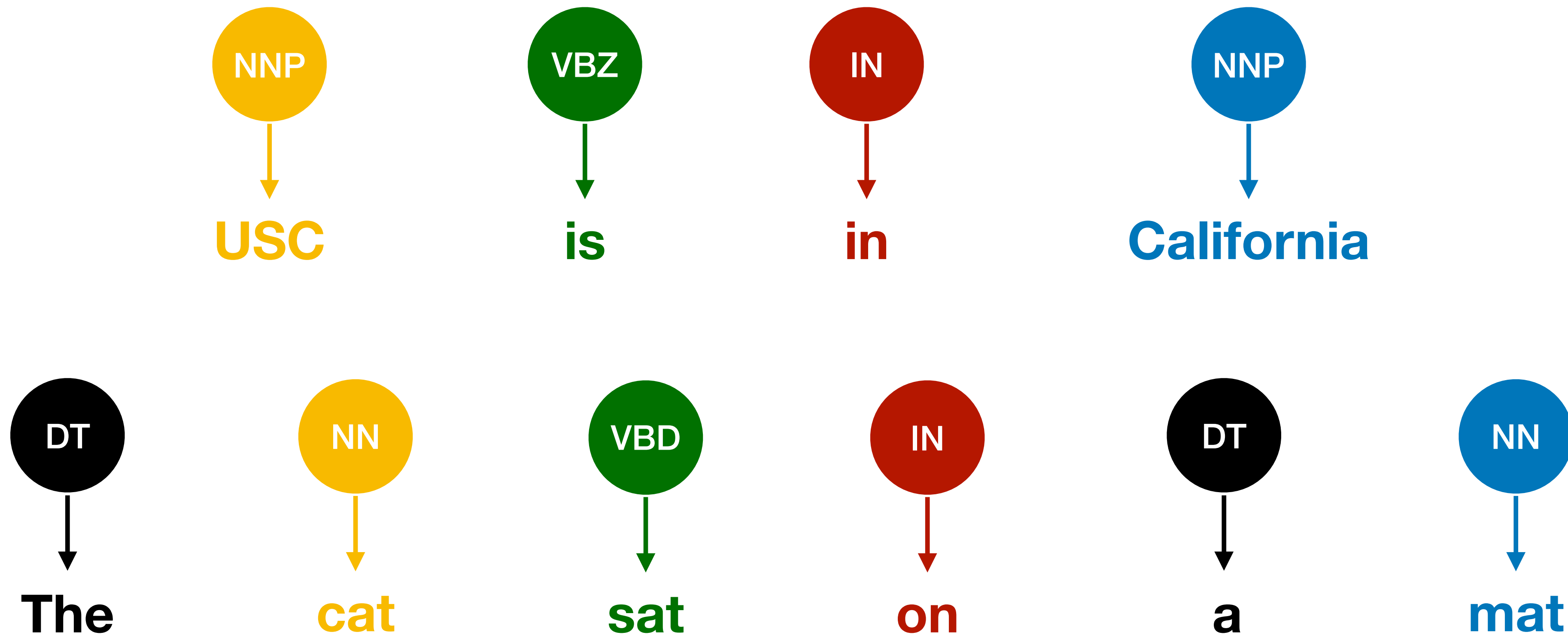


Structured Sequences



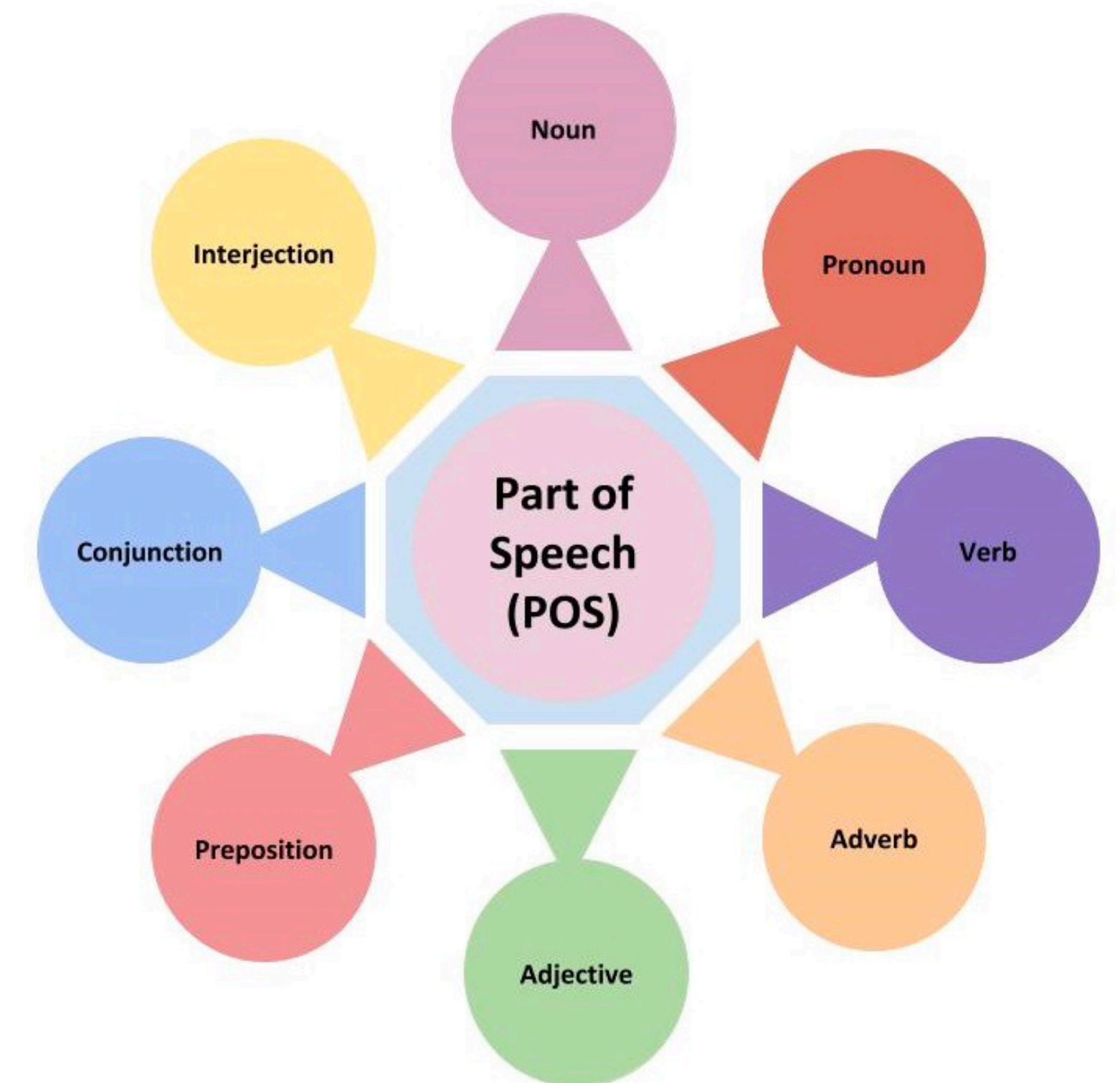
What are Part-of-Speech (POS) Tags

- Word classes or syntactic categories
- Reveal useful information about the syntactic role of a word (and its neighbors!)



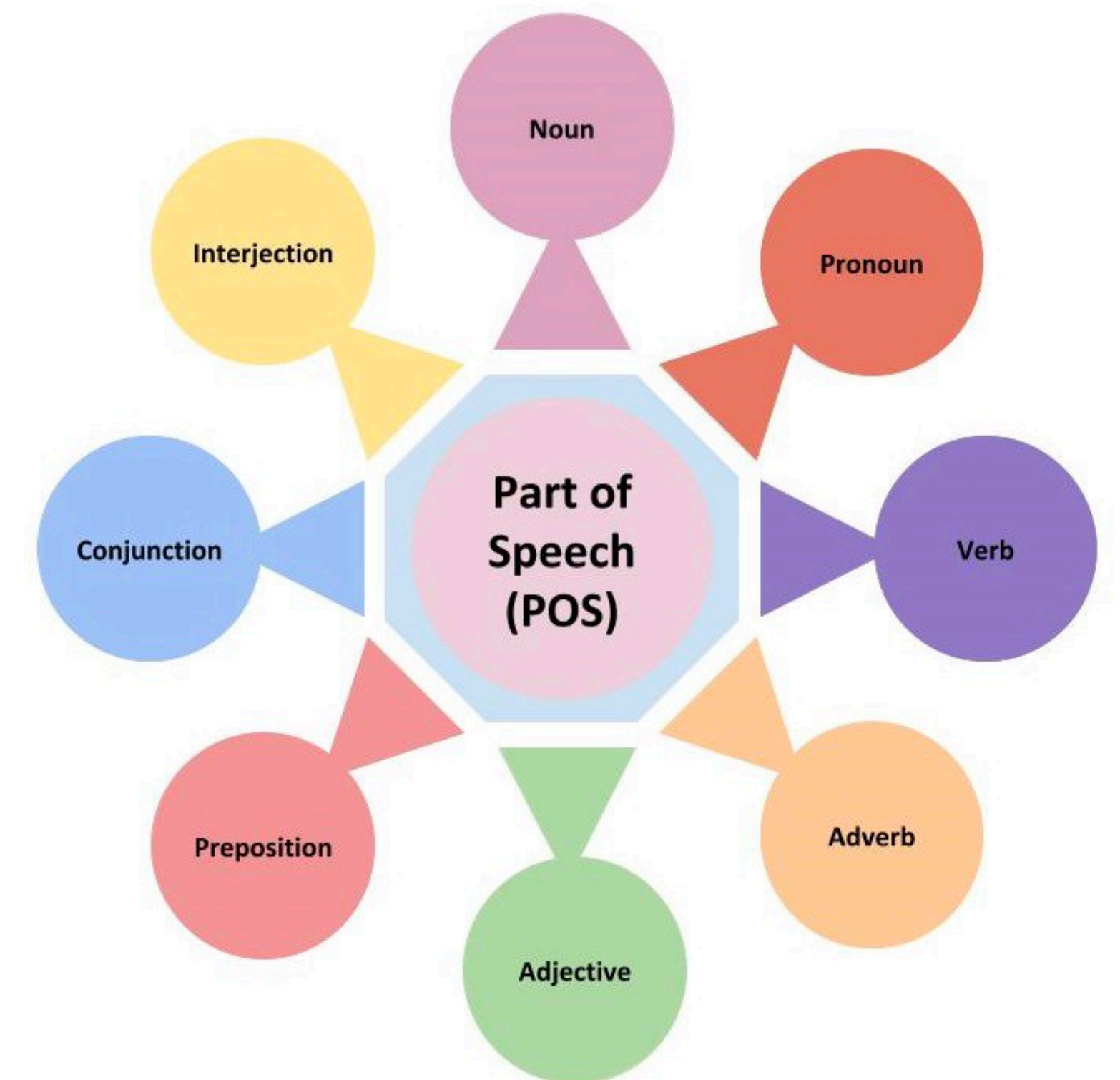
Part of Speech

- Different words have different syntactic functions
- Can be roughly divided into two classes
 - **Closed class:** fixed membership, function words
 - e.g. prepositions (in, on, of), determiners (a, the)
 - **Open class:** New words get added frequently
 - e.g. nouns (Twitter, Facebook, ChatGPT), verbs (google), adjectives and adverbs.



Part of Speech

- How many part of speech tags are there in English?
 - A. < 10
 - B. 10 - 30
 - C. 30 - 50
 - D. > 50



Penn Tree Bank Tagset

Tag	Description	Example	Tag	Description	Example	Tag	Description	Example
CC	coordinating conjunction	<i>and, but, or</i>	PDT	predeterminer	<i>all, both</i>	VBP	verb non-3sg present	<i>eat</i>
CD	cardinal number	<i>one, two</i>	POS	possessive ending	<i>'s</i>	VBZ	verb 3sg pres	<i>eats</i>
DT	determiner	<i>a, the</i>	PRP	personal pronoun	<i>I, you, he</i>	WDT	wh-determ.	<i>which, that</i>
EX	existential 'there'	<i>there</i>	PRP\$	possess. pronoun	<i>your, one's</i>	WP	wh-pronoun	<i>what, who</i>
FW	foreign word	<i>mea culpa</i>	RB	adverb	<i>quickly</i>	WP\$	wh-possess.	<i>whose</i>
IN	preposition/ subordin-conj	<i>of, in, by</i>	RBR	comparative adverb	<i>faster</i>	WRB	wh-adverb	<i>how, where</i>
JJ	adjective	<i>yellow</i>	RBS	superlatv. adverb	<i>fastest</i>	\$	dollar sign	<i>\$</i>
JJR	comparative adj	<i>bigger</i>	RP	particle	<i>up, off</i>	#	pound sign	<i>#</i>
JJS	superlative adj	<i>wildest</i>	SYM	symbol	<i>+, %, &</i>	“	left quote	<i>‘ or “</i>
LS	list item marker	<i>1, 2, One</i>	TO	“to”	<i>to</i>	”	right quote	<i>’ or ”</i>
MD	modal	<i>can, should</i>	UH	interjection	<i>ah, oops</i>	(	left paren	<i>[, (, {, <</i>
NN	sing or mass noun	<i>llama</i>	VB	verb base form	<i>eat</i>	)	right paren	<i>],), }, ></i>
NNS	noun, plural	<i>llamas</i>	VBD	verb past tense	<i>ate</i>	,	comma	<i>,</i>
NNP	proper noun, sing.	<i>IBM</i>	VBG	verb gerund	<i>eating</i>	.	sent-end punc	<i>. ! ?</i>
NNPS	proper noun, plu.	<i>Carolinas</i>	VBN	verb past part.	<i>eaten</i>	:	sent-mid punc	<i>: ; ... - -</i>

45 tags!
(Marcus et al., 1993)

The Task of Part of Speech Tagging

- Tag each word with its part of speech
- Disambiguation task: each word might have different senses/functions

- The/DT **back/ADJ** door/NN

- On/IN my/PRP\$ **back/NN**

- Win/VB the/DT voters/NNS **back/RP**

←
←
← Same word, different tags

Types:		WSJ		Brown	
Unambiguous	(1 tag)	44,432	(86%)	45,799	(85%)
Ambiguous	(2+ tags)	7,025	(14%)	8,050	(15%)
Tokens:					
Unambiguous	(1 tag)	577,421	(45%)	384,349	(33%)
Ambiguous	(2+ tags)	711,780	(55%)	786,646	(67%)

A Simple Baseline

- Many words might be easy to disambiguate
- **Most Frequent Class**: Assign each token (word) to the class it occurred most in the training data. (e.g. student/NN)
 - Entirely discarding contextual information
- How accurate do you think this baseline would be at tagging words?
 - A. < 50%
 - B. 50% - 75%
 - C. 75% - 90%
 - D. > 90%

Accurately tags **92.34%** of word tokens on Wall Street Journal (WSJ)

POS Tagging **Not** Solved!

- State of the art: $\sim 97\%$
- Sentence level accuracies
 - Average length of English sentence ~ 14 words
 - $0.92^{14} = 31\%$ vs. $0.97^{14} = 65\%$
- Highly relying on domain information
 - Training data and testing data must be from the same domain
 - $< 70\%$ on data from social media

Some Observations

- The function (or POS) of a word depends on its context
 - The/DT **back/ADJ** door/NN
 - On/IN my/PRP\$ **back/NN**
 - Win/VB the/DT voters/NNS **back/RP**
- Certain POS combinations are extremely unlikely
 - **<JJ, DT>** (“good the”) or **<DT, IN>** (“the in”)
- Better to make predictions on entire sentences instead of individual words

Sequence Labeling Models!

- Modeling $P(Y|X)$ directly

An Essential Question

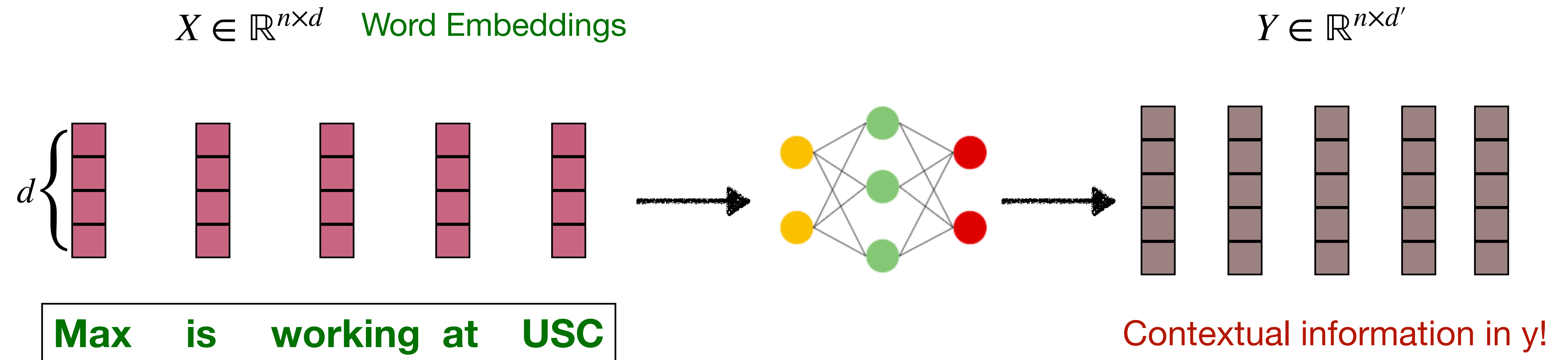
- Do we need structured models if the feature representations of the input sentence is perfect



$$P(y_1, \dots, y_n | x_1, \dots, x_n) = \prod_{j=1}^n P(y_j | x_1, \dots, x_n) \quad ?$$

Our Goal: Sentence Representations

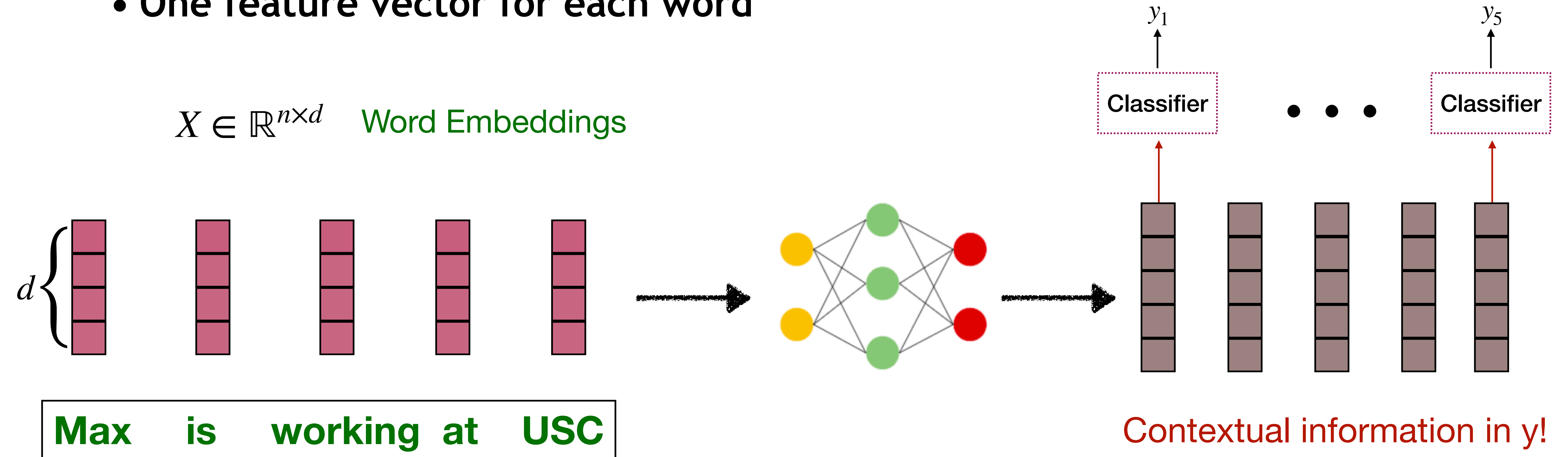
- One feature vector for each word



$$P(y_1, \dots, y_n | x_1, \dots, x_n) = \prod_{j=1}^n P(y_j | x_1, \dots, x_n)$$

Our Goal: Sentence Representations

- One feature vector for each word



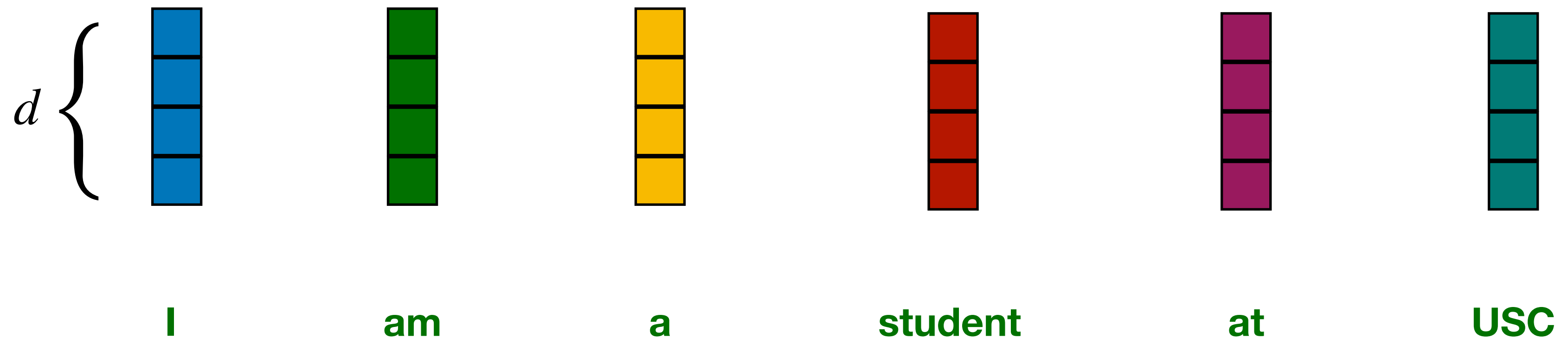
$$P(y_1, \dots, y_n | x_1, \dots, x_n) = \prod_{j=1}^n P(y_j | x_1, \dots, x_n)$$

Recap: One Vector to Represent a Document

- Classification

- We need **a single feature vector** to feed into ML classifiers

$X \in \mathbb{R}^{n \times d}$ Word Embeddings

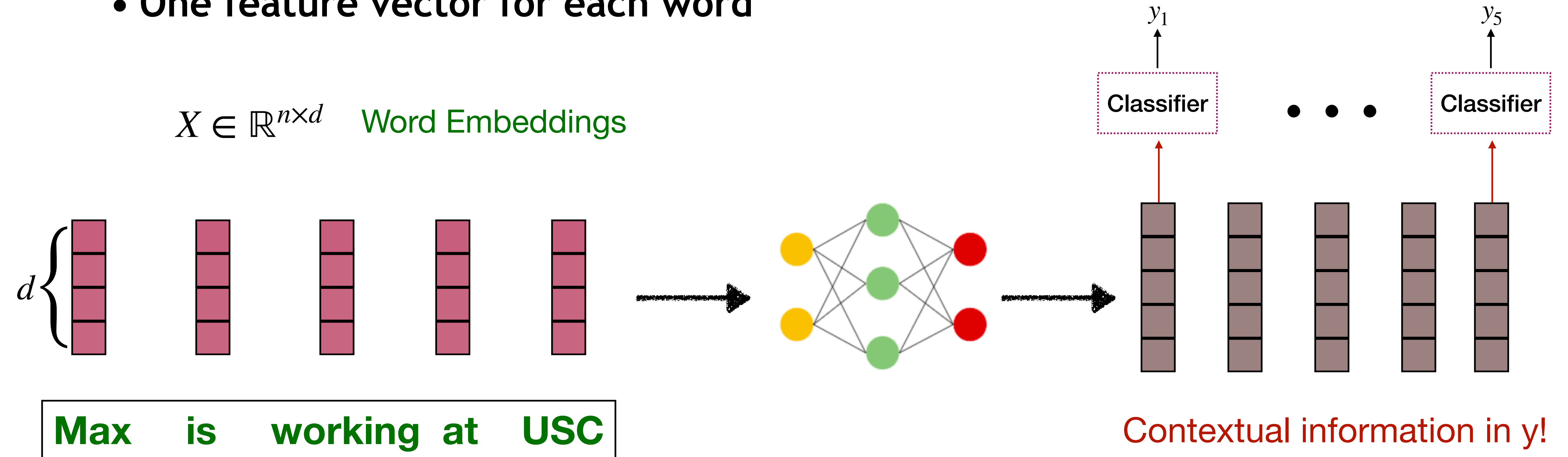


$$y = f(x_1, \dots, x_n) \in \mathbb{R}^{d'}$$



Our Goal: Sentence Representations

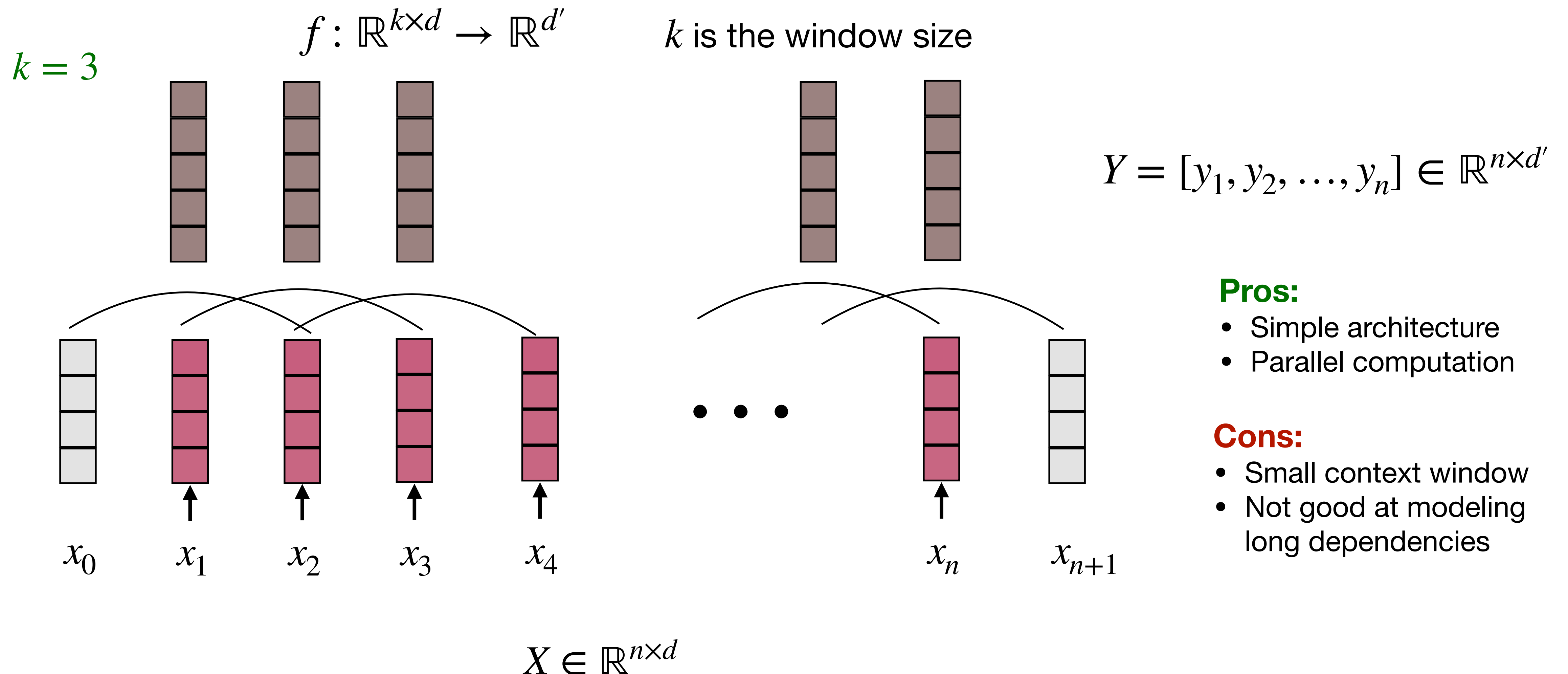
- One feature vector for each word



$$P(y_1, \dots, y_n | x_1, \dots, x_n) = \prod_{j=1}^n P(y_j | x_1, \dots, x_n)$$

Convolutional Neural Networks (CNNs)

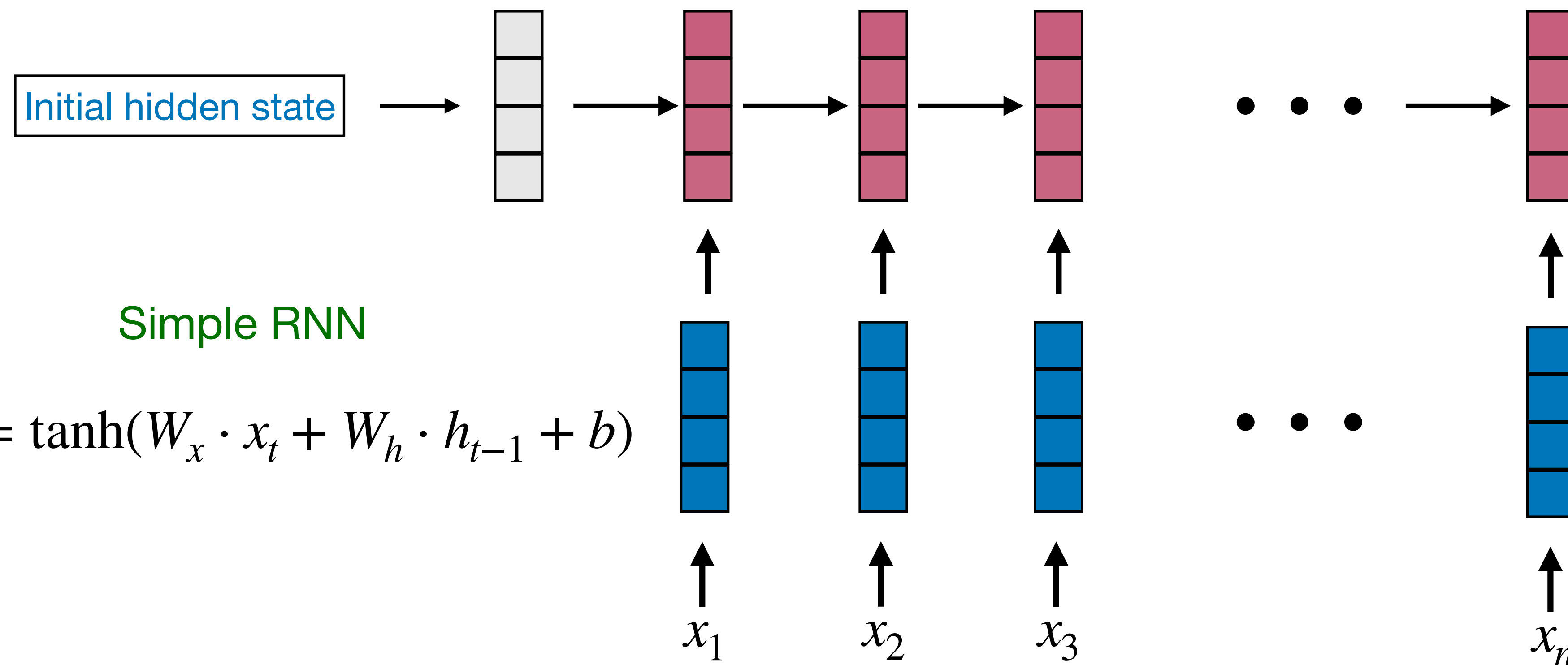
- **Basic Idea:** only model a segment of input with **fixed window-size**



Recurrent Neural Networks

Neural Networks for Sentence Representations

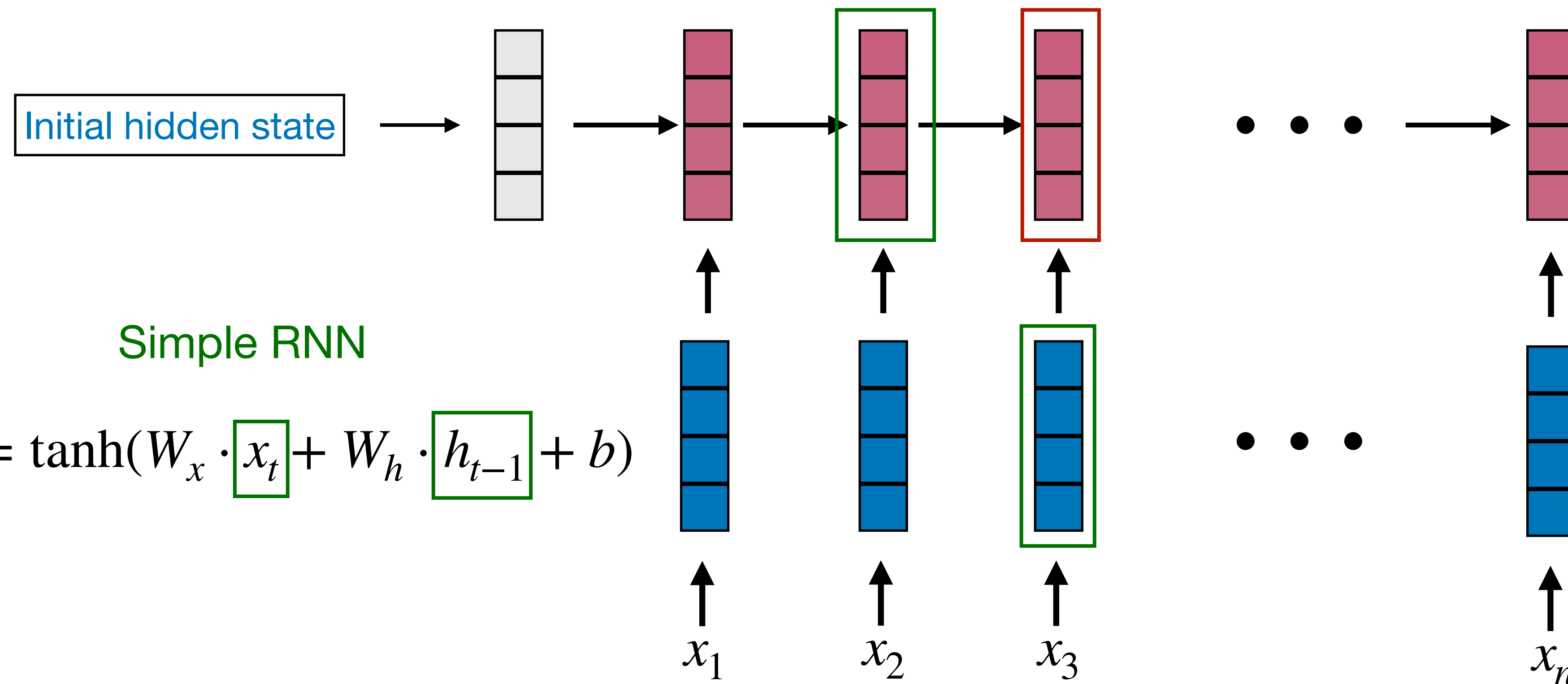
- Recurrent Neural Networks (RNNs)
 - Re-using one feed forward network in a recurrent way



Neural Networks for Sentence Representations

- Recurrent Neural Networks (RNNs)
 - Re-using one feed forward network in a recurrent way

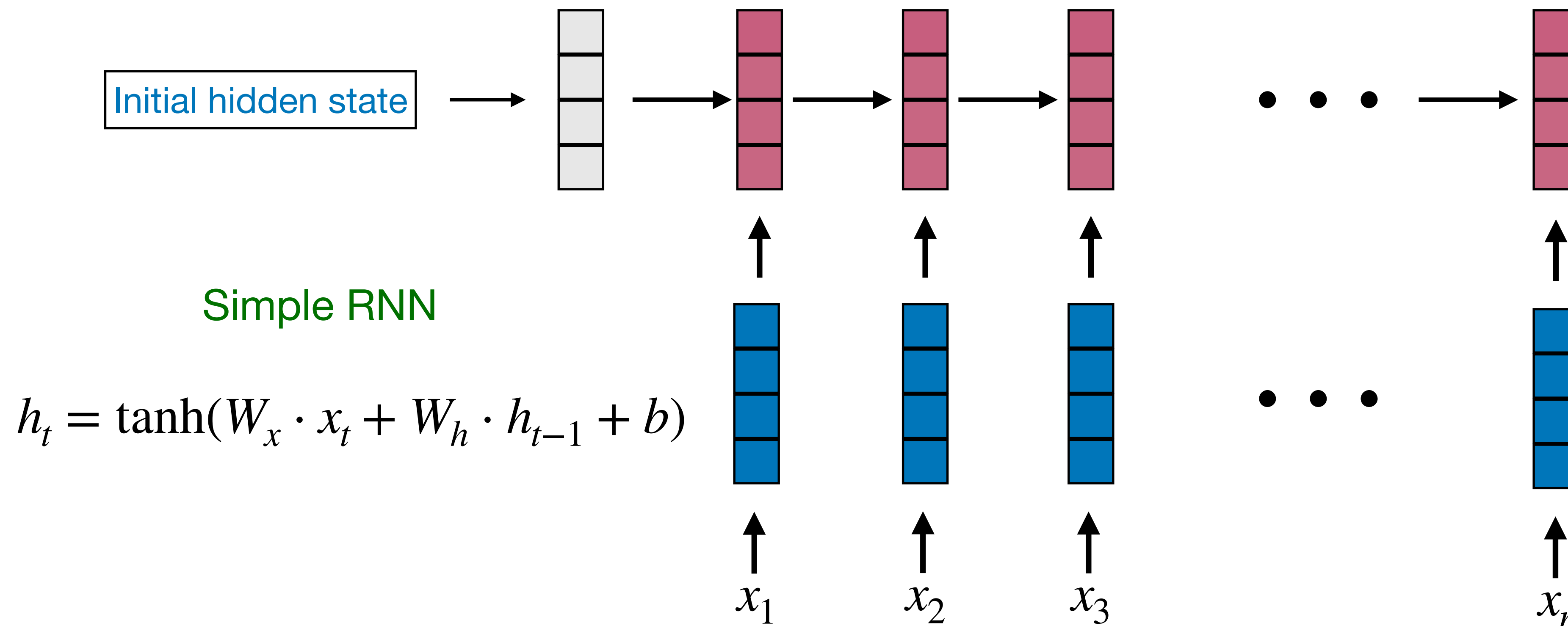
Inherent Markov Property



Problems of Simple RNN

- No future contexts
- Hard to train
 - Gradient vanishing/exploding

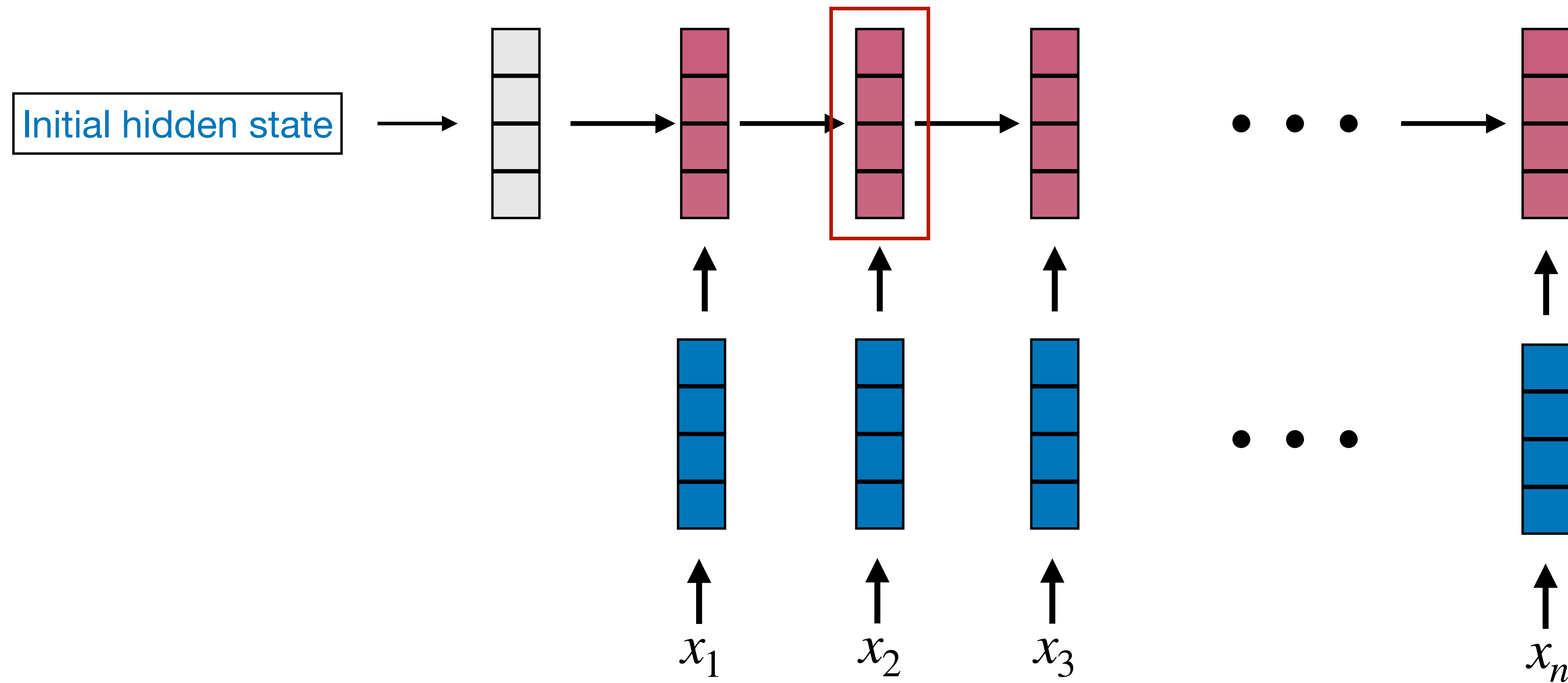
- Inefficient
 - Sequential computation
- Limited size of hidden states
 - Memory cost



Bidirectional RNNs

- RNN **cannot** model **future** information

h_2 cannot access the information in $x_3, \dots, x_n$



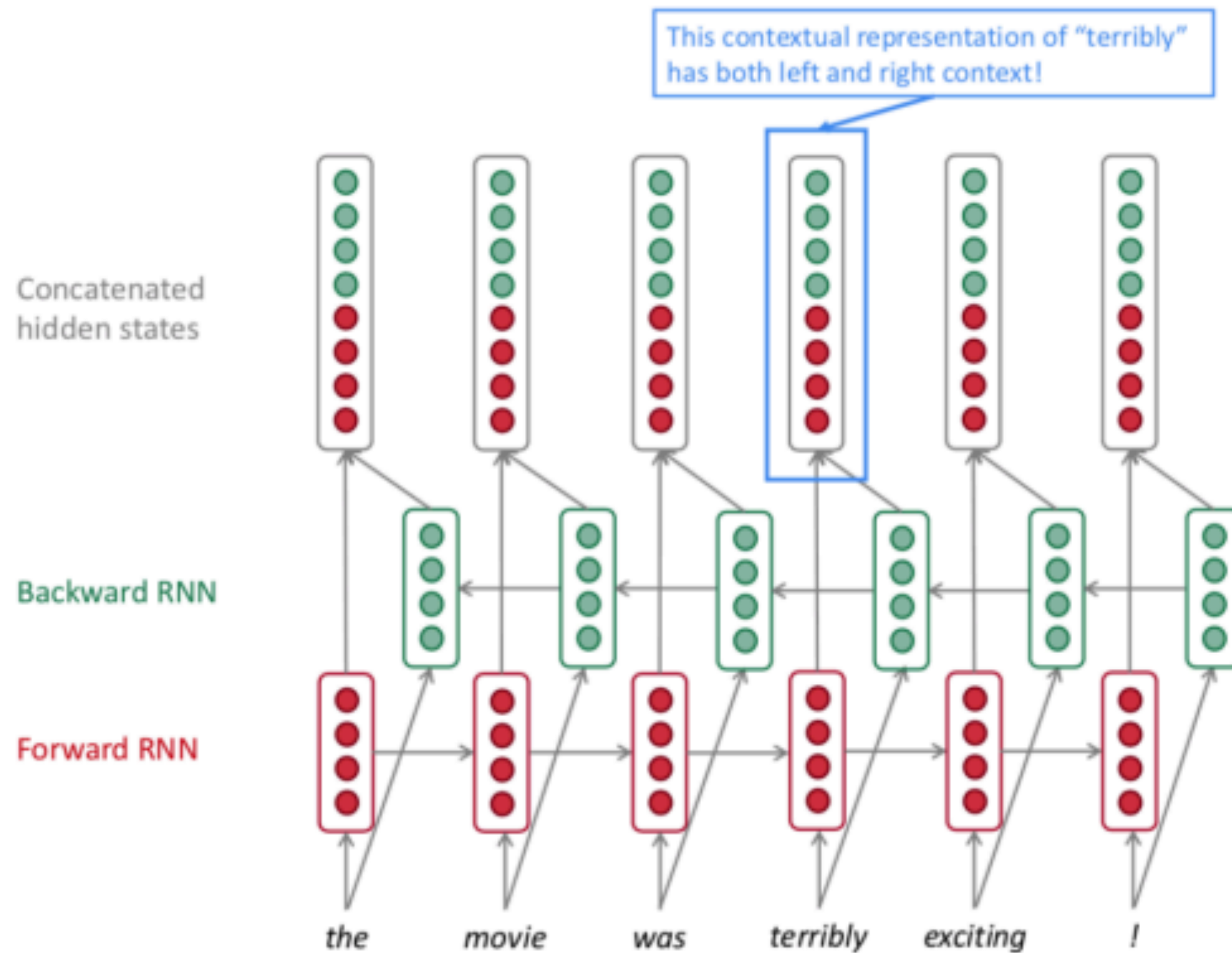
Bidirectional RNNs

- Future information is important for sequence labeling tasks!

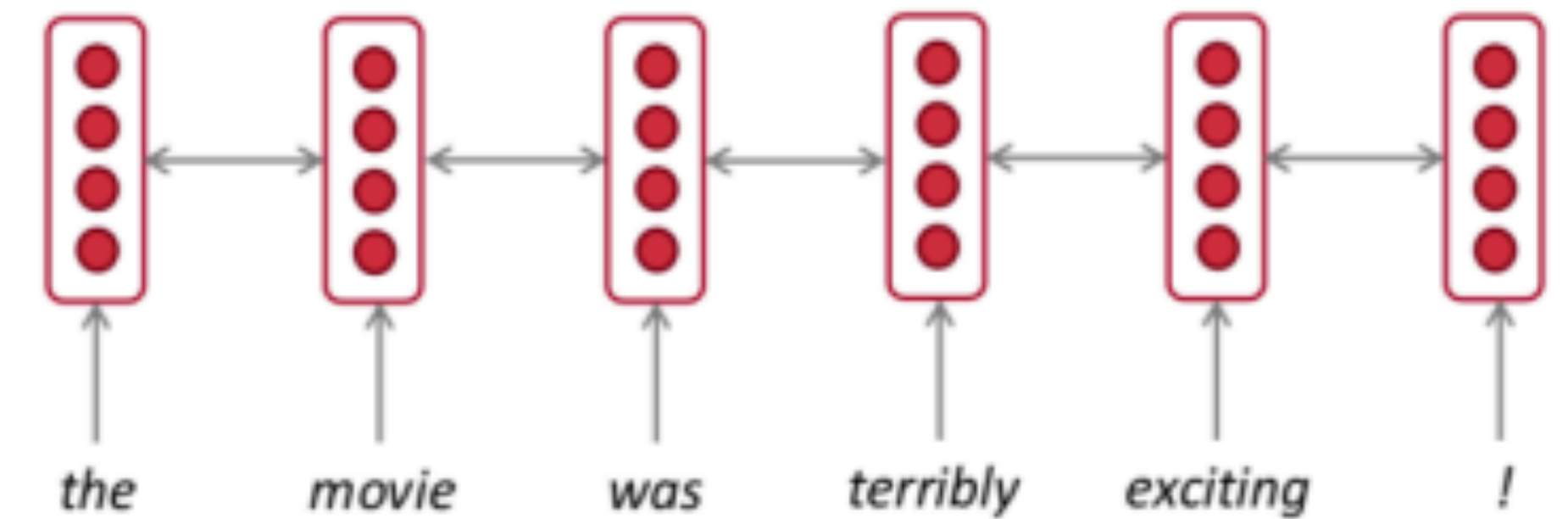


$$P(y_1, \dots, y_n | x_1, \dots, x_n) = \prod_{j=1}^n P(y_j | x_1, \dots, x_n)$$

Bidirectional RNNs



=



$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t) \in \mathbb{R}^h$$

$$\vec{\mathbf{h}}_t = f_1(\vec{\mathbf{h}}_{t-1}, \mathbf{x}_t), t = 1, 2, \dots, n$$

$$\overleftarrow{\mathbf{h}}_t = f_2(\overleftarrow{\mathbf{h}}_{t+1}, \mathbf{x}_t), t = n, n-1, \dots, 1$$

$$\mathbf{h}_t = [\overleftarrow{\mathbf{h}}_t, \vec{\mathbf{h}}_t] \in \mathbb{R}^{2h}$$

Bidirectional RNNs

- Bidirectional RNNs are only applicable if we have access to the **entire input sequence**
- If we do have entire input sequence, bidirectionality is powerful (and should be the **default choice**)
- A very common choice for sentence/document encoding: multi-layer bidirectional RNNs

Problems of Simple RNN

- No future contexts

- Hard to train

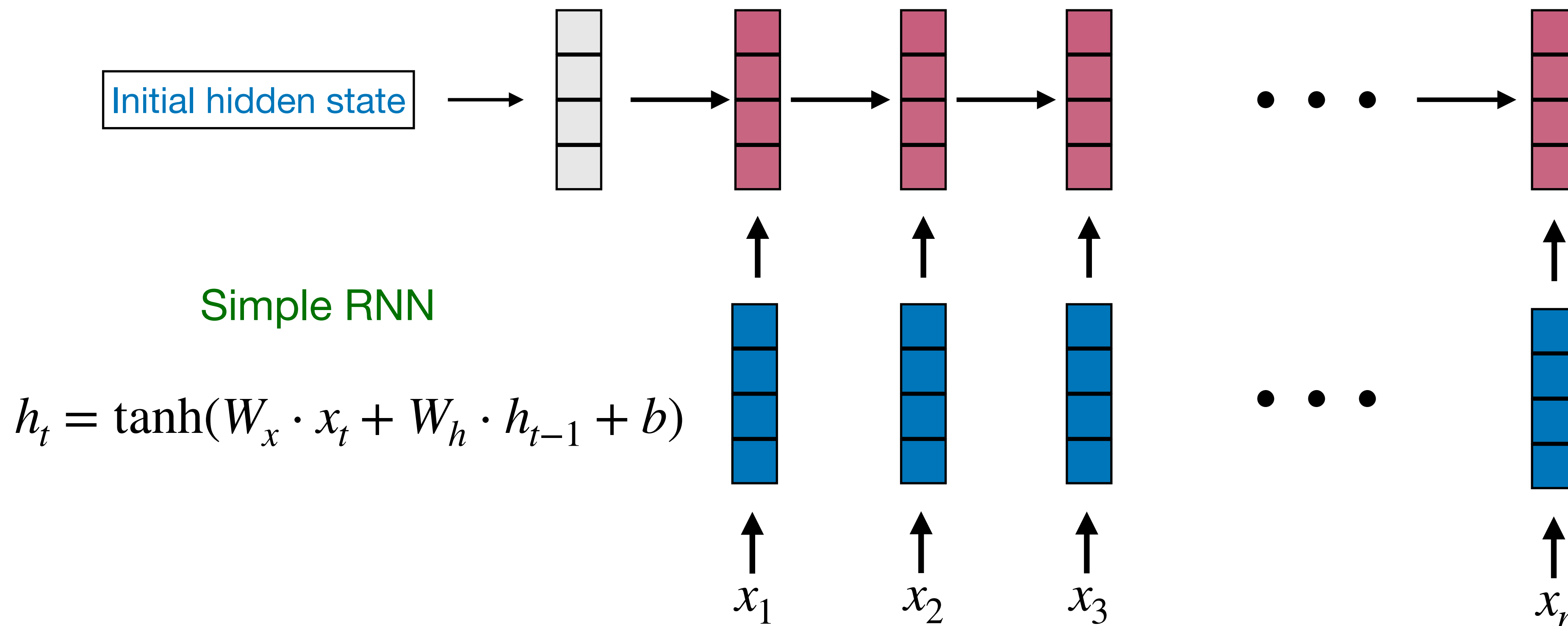
- Gradient vanishing/exploding

- Inefficient

- Sequential computation

- Limited size of hidden states

- Memory cost



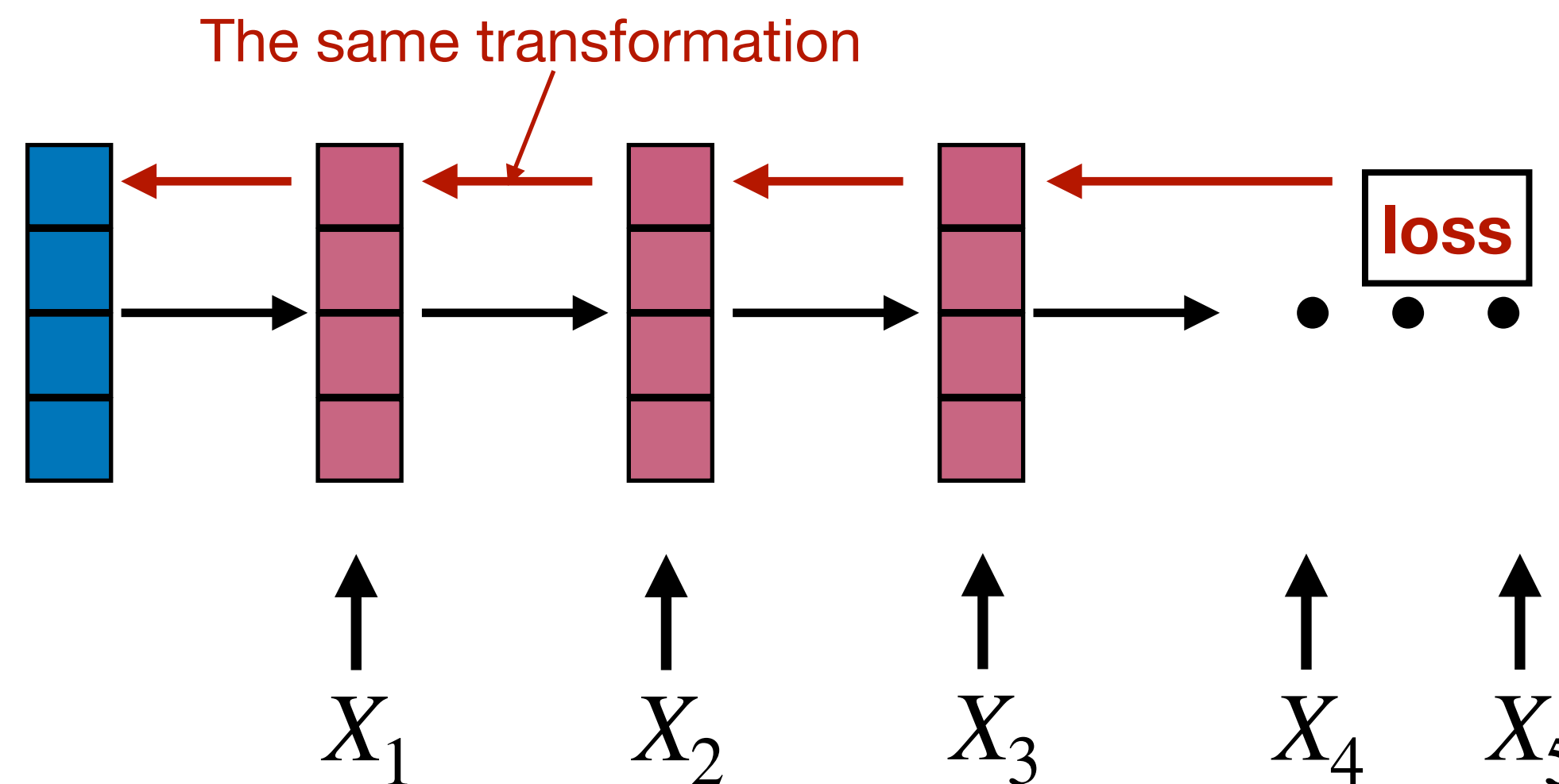
Simple RNN is hard to train

- Hard to capture long-distance information: **vanishing/exploding gradients**

$$h_t = \tanh(W_x \cdot x_t + W_h \cdot h_{t-1} + b)$$

$$\left\| \frac{\partial h_t}{\partial h_{t-1}} \right\| \sim \|W_h\|$$

$$\left\| \frac{\partial h_t}{\partial h_{t-m}} \right\| \sim \|W_h\|^m$$



Why is this not a serious problem for multi-layer FFN/CNN?

Advanced RNN Variants

- Long-short Term Memory (LSTMs)
- Gated Recurrent Units (GRUs)

LSTMs

$$\mathbf{i}_t = \sigma(\mathbf{W}^i \mathbf{h}_{t-1} + \mathbf{U}^i \mathbf{x}_t + \mathbf{b}^i)$$

$$\mathbf{f}_t = \sigma(\mathbf{W}^f \mathbf{h}_{t-1} + \mathbf{U}^f \mathbf{x}_t + \mathbf{b}^f)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}^o \mathbf{h}_{t-1} + \mathbf{U}^o \mathbf{x}_t + \mathbf{b}^o)$$

$$\mathbf{g}_t = \tanh(\mathbf{W}^g \mathbf{h}_{t-1} + \mathbf{U}^g \mathbf{x}_t + \mathbf{b}^g)$$

$$\mathbf{c}_t = \mathbf{c}_{t-1} \odot \mathbf{f}_t + \mathbf{g}_t \odot \mathbf{i}_t$$

$$\mathbf{h}_t = \tanh(\mathbf{c}_t) \odot \mathbf{o}_t$$

GRUs

$$\mathbf{r}_t = \sigma(\mathbf{W}^r \mathbf{h}_{t-1} + \mathbf{U}^r \mathbf{x}_t + \mathbf{b}^r)$$

$$\mathbf{z}_t = \sigma(\mathbf{W}^z \mathbf{h}_{t-1} + \mathbf{U}^z \mathbf{x}_t + \mathbf{b}^z)$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{U} \mathbf{x}_t + \mathbf{b})$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$$

LSTM is more an art than a science

LSTM: Long Short-Term Memory

$$\mathbf{i}_t = \sigma(\mathbf{W}^i \mathbf{h}_{t-1} + \mathbf{U}^i \mathbf{x}_t + \mathbf{b}^i)$$

Input gate: $i_t \in (0,1)$

$$\mathbf{f}_t = \sigma(\mathbf{W}^f \mathbf{h}_{t-1} + \mathbf{U}^f \mathbf{x}_t + \mathbf{b}^f)$$

Forget gate: $f_t \in (0,1)$

$$\mathbf{o}_t = \sigma(\mathbf{W}^o \mathbf{h}_{t-1} + \mathbf{U}^o \mathbf{x}_t + \mathbf{b}^o)$$

Output gate: $o_t \in (0,1)$

$$\mathbf{g}_t = \tanh(\mathbf{W}^g \mathbf{h}_{t-1} + \mathbf{U}^g \mathbf{x}_t + \mathbf{b}^g)$$

Simple RNN

$$\mathbf{c}_t = \mathbf{c}_{t-1} \odot \mathbf{f}_t + \mathbf{g}_t \odot \mathbf{i}_t$$

Cell state vector (internal memory)

$$\mathbf{h}_t = \tanh(\mathbf{c}_t) \odot \mathbf{o}_t$$

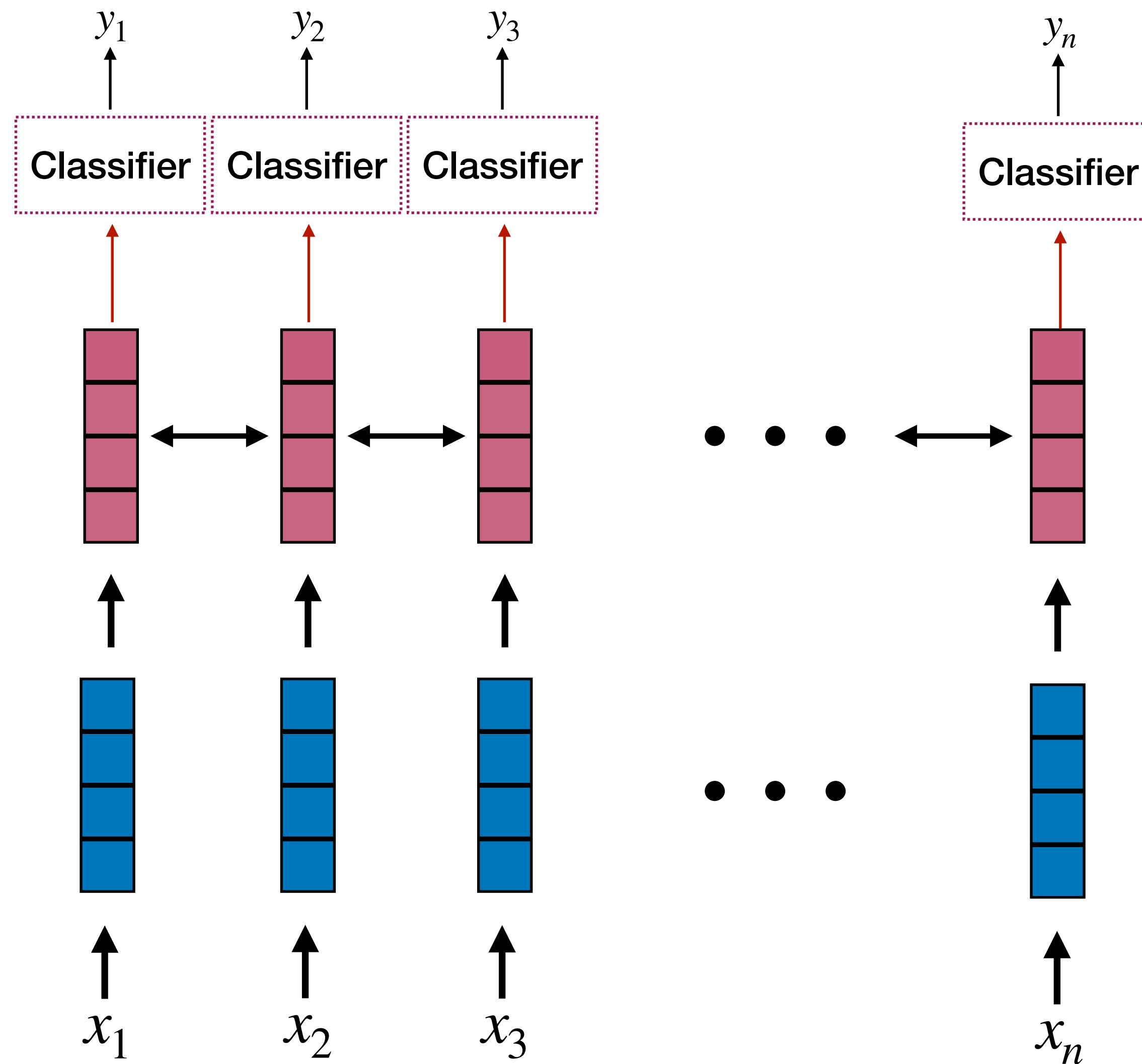
Final hidden vector

All the components are directly learned during training

LSTMs for Sequence Labeling

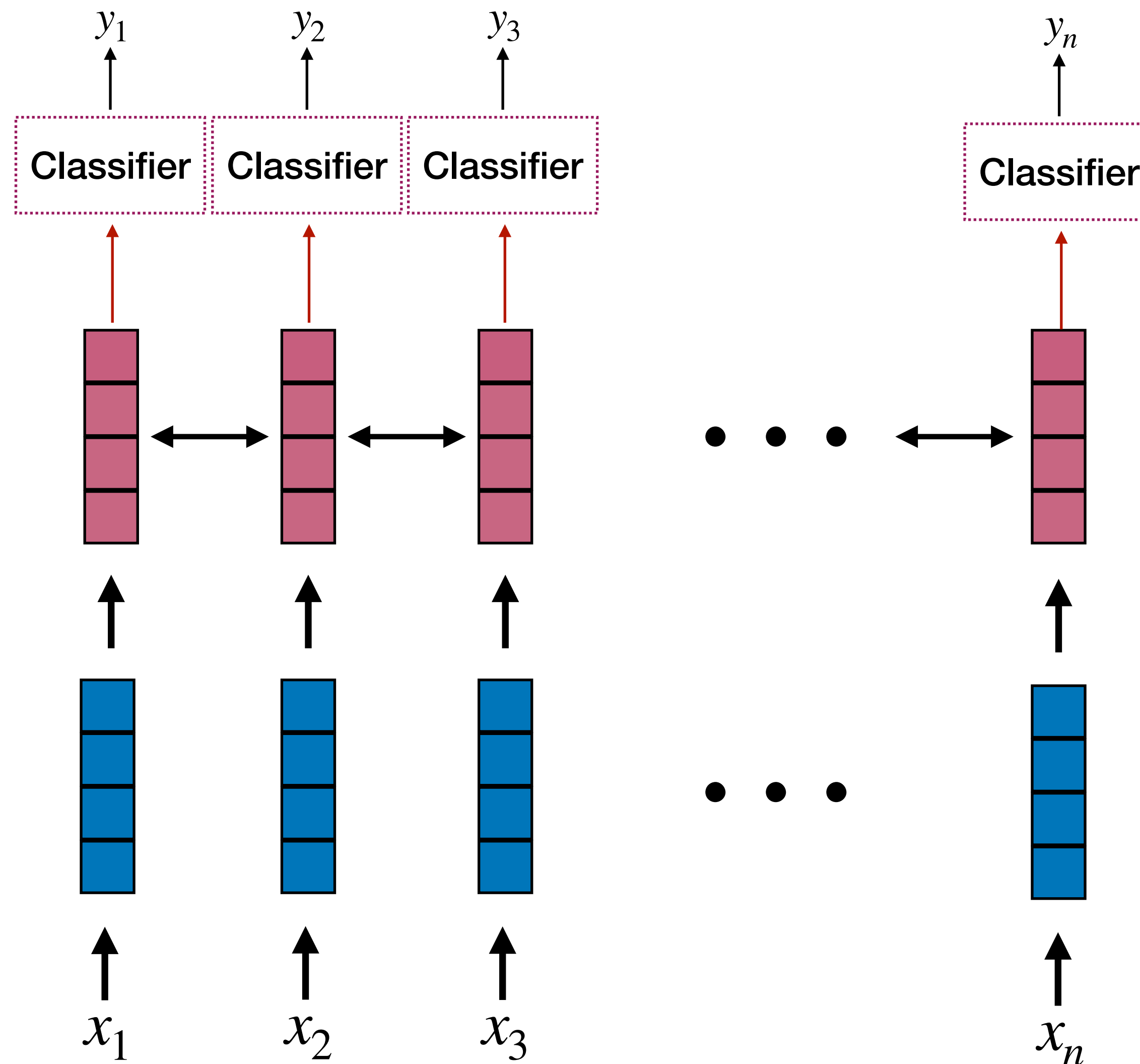
LSTM for Sequence Labeling

- A simple bidirectional LSTM model



Sequence Labeling

- How to initialize parameters?



LSTM: $\text{Unif}(-\sqrt{1/d}, \sqrt{1/d})$

Embedding: Word2Vec or GloVe

Sequence Labeling

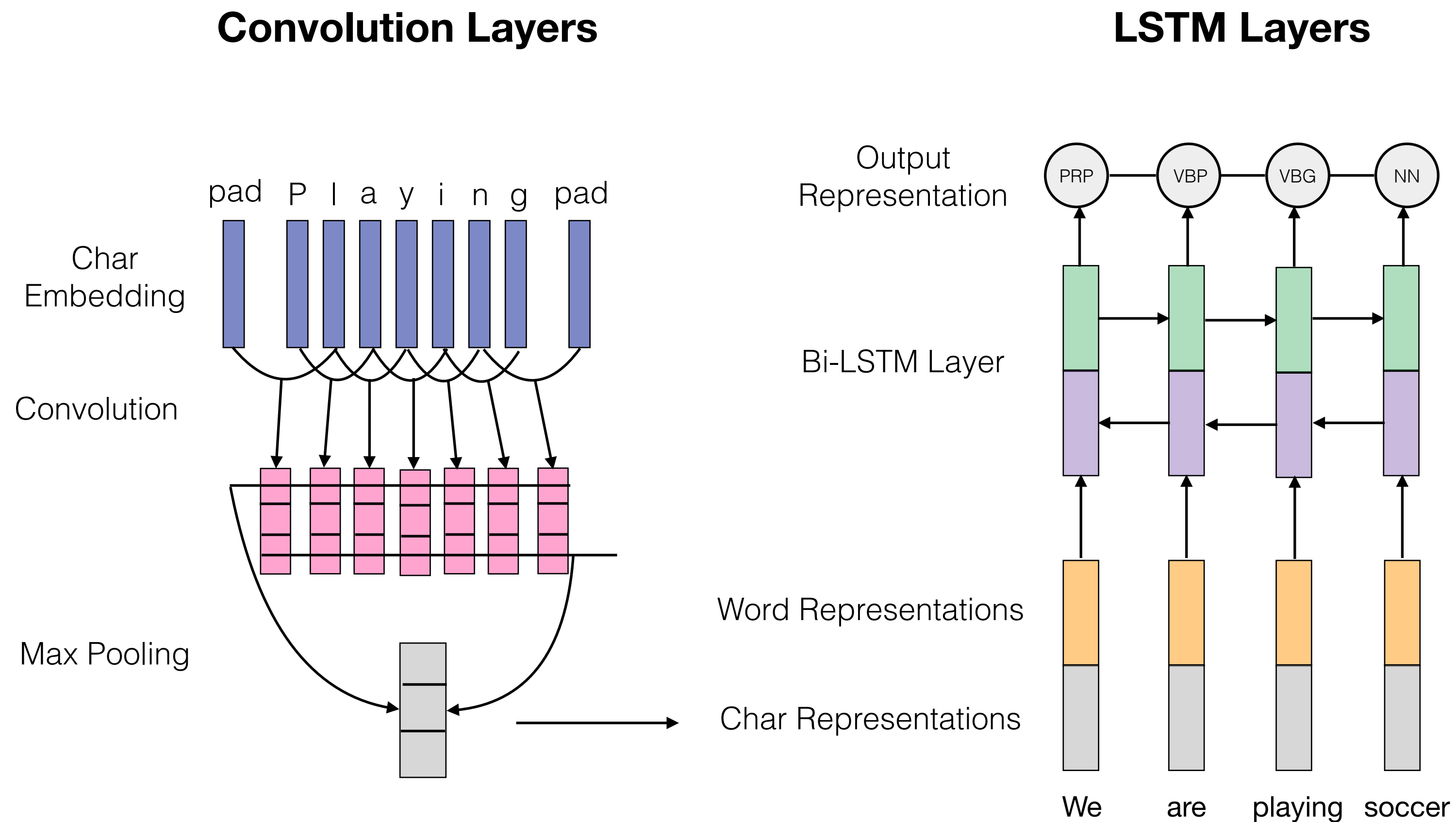
- A simple bidirectional LSTM model

	POS Tagging	NER
Classic SoTA	97.3%	91.2%
BLSTM-Random	96.1%	80.7%
BLSTM-GloVe	96.9%	87.0%

BLSTM is not good enough!

Bidirectional LSTMs + CNNs

- Bidirectional LSTM only encodes word-level information
- Spelling features are important
 - Using CNN to model character-level information



Sequence Labeling

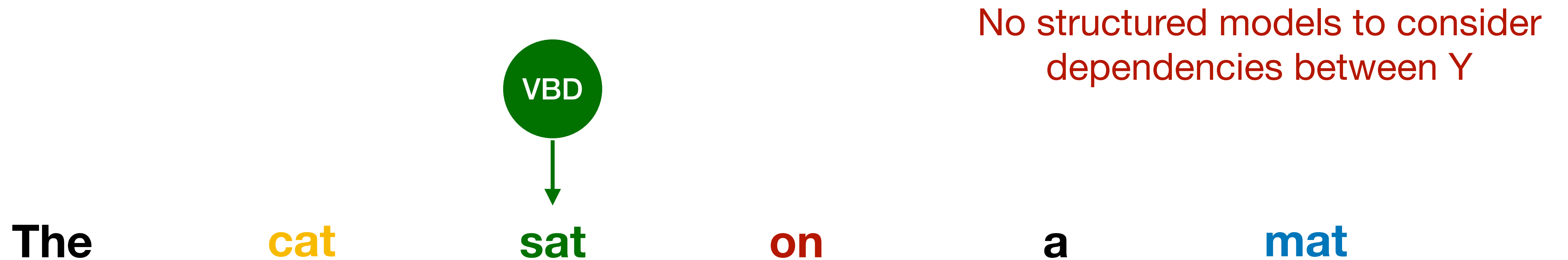
- **BLSTM-CNN**

	POS Tagging	NER
Classic SoTA	97.3%	91.2%
BLSTM	96.9%	87.0%
BLSTM-CNN	97.3%	89.4%

BLSTM-CNN is closed to previous SoTA!

An Essential Question

- Do we need structured models if the feature representations of the input sentence is perfect



$$P(y_1, \dots, y_n | x_1, \dots, x_n) = \prod_{j=1}^n P(y_j | x_1, \dots, x_n) \quad ?$$

BiLSTM + Char-level CNN is not strong enough...

Combining structured models with LSTM?

Log-Linear Models

- ▶ We have some input domain $\mathcal{X}$, and a finite label set $\mathcal{Y}$. Aim is to provide a conditional probability $p(y \mid x)$ for any $x \in \mathcal{X}$ and $y \in \mathcal{Y}$.
- ▶ A feature is a function $f : \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R}$
(Often binary features or indicator functions $f_k : \mathcal{X} \times \mathcal{Y} \rightarrow \{0, 1\}$).
- ▶ Say we have m features f_k for $k = 1 \dots m$
 $\Rightarrow$ A feature vector $f(x, y) \in \mathbb{R}^m$ for any $x \in \mathcal{X}$ and $y \in \mathcal{Y}$.
- ▶ We also have a **parameter vector** $v \in \mathbb{R}^m$
- ▶ We define

$$p(y \mid x; v) = \frac{e^{v \cdot f(x, y)}}{\sum_{y' \in \mathcal{Y}} e^{v \cdot f(x, y')}}$$

Why the name?

$$p(y \mid x; v) = \frac{e^{v \cdot f(x, y)}}{\sum_{y' \in \mathcal{Y}} e^{v \cdot f(x, y')}}$$

$$\log p(y \mid x; v) = \underbrace{v \cdot f(x, y)}_{\text{Linear term}} - \underbrace{\log \sum_{y' \in \mathcal{Y}} e^{v \cdot f(x, y')}}_{\text{Normalization term}}$$

Features in Log-linear Models

$S = S_1, S_2, \dots, S_n$

NNP

VBZ

IN

NNP

$X = X_1, X_2, \dots, X_n$

USC

is

in

California

Theoretically, we can use any features in X and S : $f(X, S)$

- The current word: **<is>**
- The surrounding words: **<USC, is>**, ...
- The current POS tag: **<VBZ>**
- The surrounding tags: **<NNP, VBZ>**, ...
- The current word and tag: **<is, VBZ>**

Feature Sparsity Problem

$$S = S_1, S_2, \dots, S_n$$



$$X = X_1, X_2, \dots, X_n$$

USC

is

in

California

Theoretically, we can use any features in X and S : $f(X, S)$

- The current word: **<is>**
- The surrounding words: **<USC, is>**, ...
- The current POS tag: **<VBZ>**
- The surrounding tags: **<NNP, VBZ>**, ...
- The current word and tag: **<is, VBZ>**

Number of Features

V

V^2

VK

Features vs. Independence

- We need independence assumptions to compute the nominator
- Stronger assumptions lead to less flexible features

$$p(y \mid x; v) = \frac{e^{v \cdot f(x, y)}}{\sum_{y' \in \mathcal{Y}} e^{v \cdot f(x, y')}} \quad \text{[denominator boxed]}$$

Maximum Entropy Markov Models (MEMMs)

- Goal: modeling the distribution

$$p(s_1, \dots, s_m \mid x_1, \dots, x_m)$$

- Markov Assumption on S

$$p(s_1, \dots, s_m \mid x_1, \dots, x_m) = \prod_{j=1}^m p(s_j \mid s_1, \dots, s_{j-1}, x_1, \dots, x_m)$$

chain rule (no assumptions)

$$= \prod_{j=1}^m p(s_j \mid s_{j-1}, x_1, \dots, x_m)$$

Markov assumption

Maximum Entropy Markov Models (MEMMs)

- We then model each term using a log-linear model:

$$p(s_j | s_{j-1}, x_1, \dots, x_m) = \frac{\exp(v \cdot f(x_1, \dots, x_m, i, s_{j-1}, s_j))}{\sum_{s'_j \in S} \exp(v \cdot f(x_1, \dots, x_m, i, s_{j-1}, s'_j))}$$

- Here $f(x_1, \dots, x_m, j, s, s')$ is the feature vector:
 - $x_1, \dots, x_m$ is the sequence of words to be tagged
 - j is the position to be tagged (any value from $1, \dots, m$)
 - s is the previous state
 - s' is the current state

Maximum Entropy Markov Models (MEMMs)

- We then model each term using a log-linear model:

$$p(s_j | s_{j-1}, x_1, \dots, x_m) = \frac{\exp(v \cdot f(x_1, \dots, x_m, i, s_{j-1}, s_j))}{\sum_{s'_j \in S} \exp(v \cdot f(x_1, \dots, x_m, i, s_{j-1}, s'_j))}$$

Practical

- Here $f(x_1, \dots, x_m, j, s, s')$ is the feature vector:
 - $x_1, \dots, x_m$ is the sequence of words to be tagged
 - j is the position to be tagged (any value from $1, \dots, m$)
 - s is the previous state
 - s' is the current state

The whole sequence of X
Only two successive tags

Features in MEMMs

$$S = S_1, S_2, \dots, S_n$$

$$X = X_1, X_2, \dots, X_n$$



USC

is

in

California

What are the most important features $f(x_1, \dots, x_m, j, s, s')$?

- The current word: **<is>**
- The surrounding words: **<USC, is>**, ...
- The current POS tag: **<VBZ>**
- The previous tag: **<NNP, VBZ>**, ...
- The current word and tag: **<is, VBZ>**
- Prefix or suffix features: **<ing>**
- ...

Conditional Random Fields (CRFs)

- Goal: modeling the distribution

$$p(s_1, \dots, s_m \mid x_1, \dots, x_m)$$

- In MEMMs we had

$$p(s_1, \dots, s_m \mid x_1, \dots, x_m) = \prod_{j=1}^m p(s_j \mid s_1, \dots, s_{j-1}, x_1, \dots, x_m)$$

chain rule (no assumptions)

$$= \prod_{j=1}^m p(s_j \mid s_{j-1}, x_1, \dots, x_m)$$

Markov assumption

- Using log-linear model

$$p(s_j \mid s_{j-1}, x_1, \dots, x_m) = \frac{\exp(v \cdot f(x_1, \dots, x_m, i, s_{j-1}, s_j))}{\sum_{s' \in \mathcal{S}} \exp(v \cdot f(x_1, \dots, x_m, i, s_{j-1}, s'_j))}$$

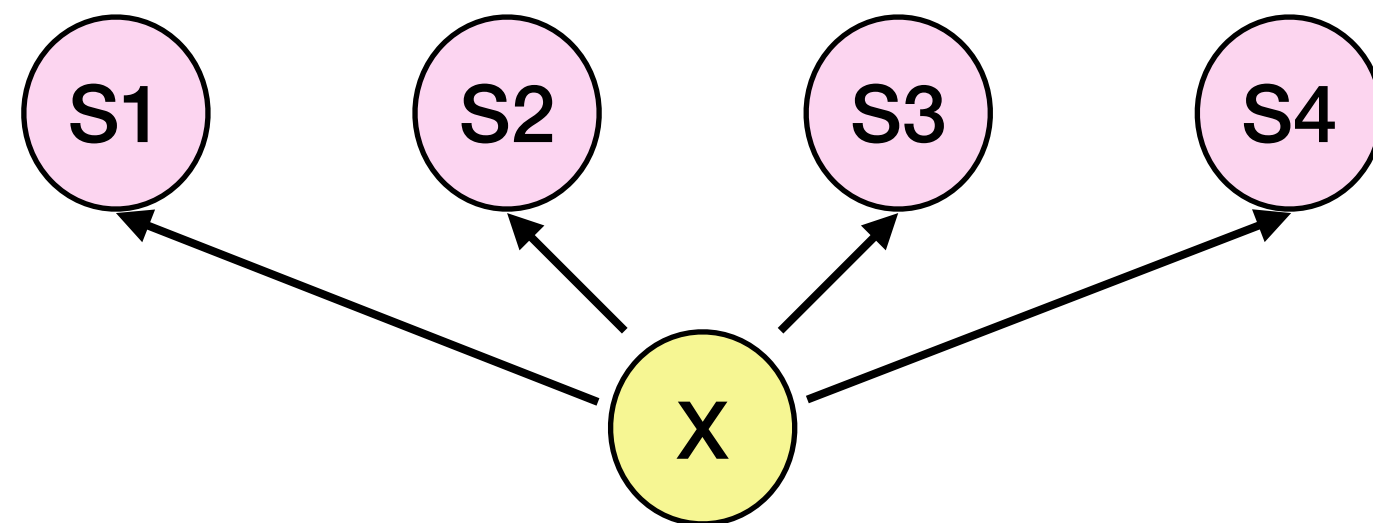
Can we build a *giant* log-linear model?

$$p(s_1, \dots, s_m \mid x_1, \dots, x_m)$$

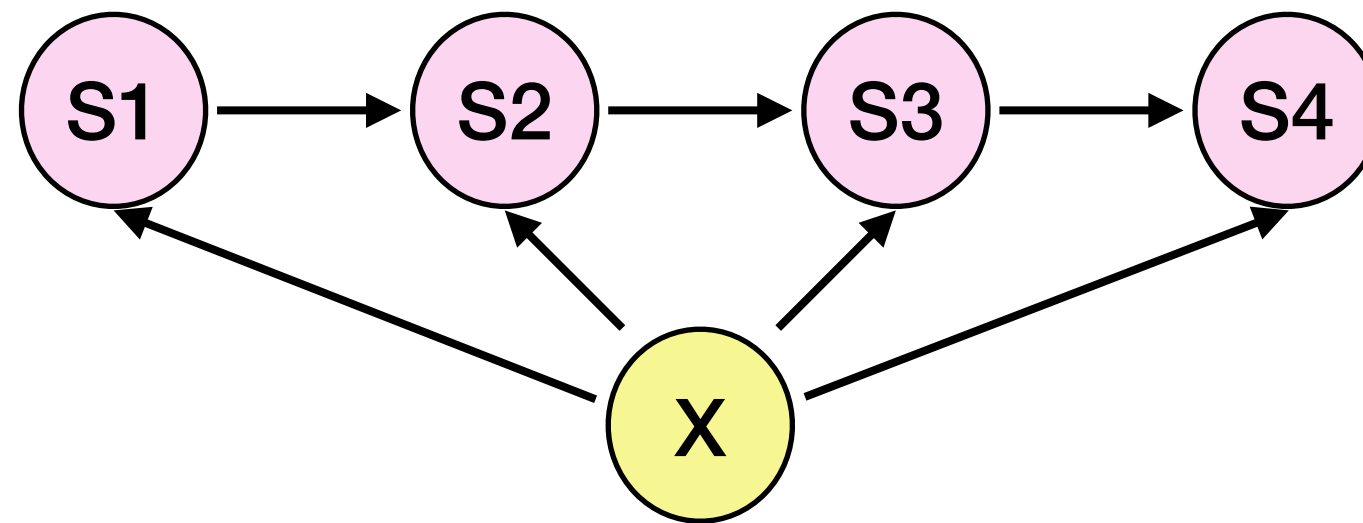
Conditional Random Fields (CRFs)

- Globally Normalized Model

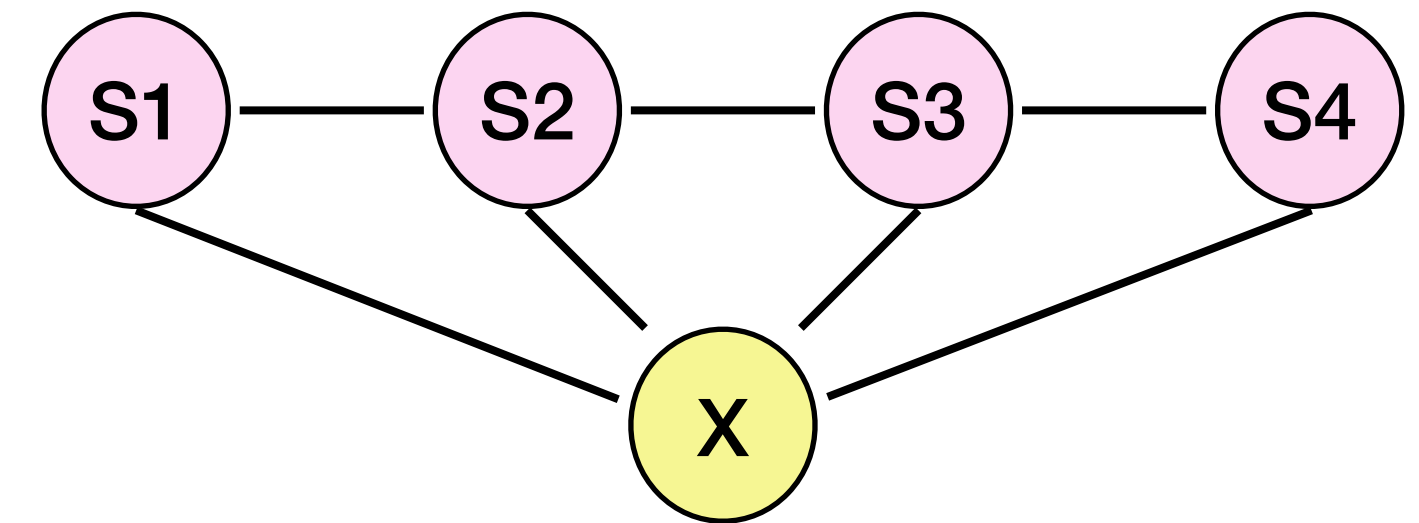
$$p(s_1, \dots, s_m | x_1, \dots, x_m) = \frac{\prod_{j=1}^m \exp(v \cdot f(x_1, \dots, x_m, i, s_{j-1}, s_j))}{\sum_{s'_1, \dots, s'_m \in \mathcal{S}} \prod_{j=1}^m \exp(v \cdot f(x_1, \dots, x_m, i, s'_{j-1}, s'_j))}$$



Individual Classification



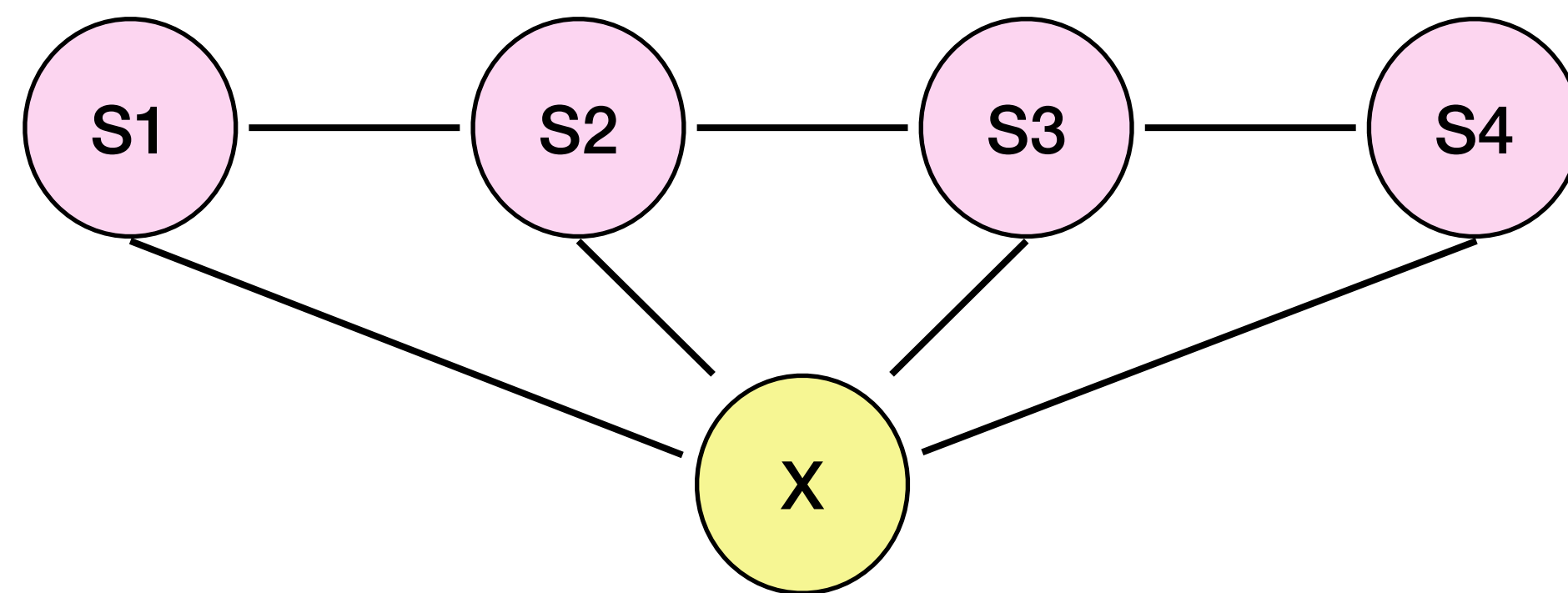
MEMM



CRF

Independence Assumptions in CRFs

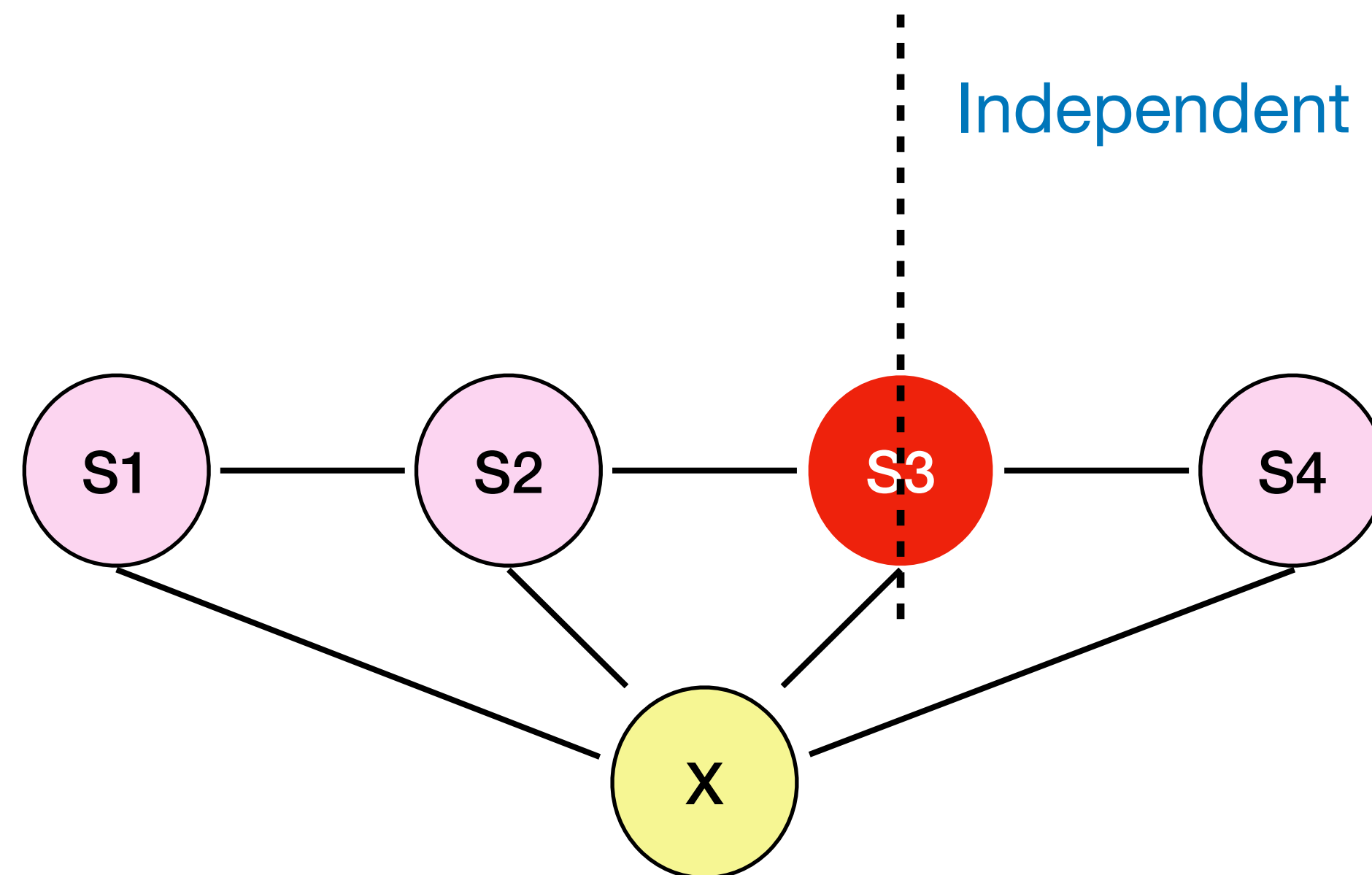
$$p(s_1, \dots, s_m | x_1, \dots, x_m) = \frac{\prod_{j=1}^m \exp(v \cdot f(x_1, \dots, x_m, i, s_{j-1}, s_j))}{\sum_{s'_1, \dots, s'_m \in \mathcal{S}} \prod_{j=1}^m \exp(v \cdot f(x_1, \dots, x_m, i, s'_{j-1}, s'_j))}$$



CRF

Independence Assumptions in CRFs

$$p(s_1, \dots, s_m | x_1, \dots, x_m) = \frac{\prod_{j=1}^m \exp(v \cdot f(x_1, \dots, x_m, i, s_{j-1}, s_j))}{\sum_{s'_1, \dots, s'_m \in \mathcal{S}} \prod_{j=1}^m \exp(v \cdot f(x_1, \dots, x_m, i, s'_{j-1}, s'_j))}$$



CRF

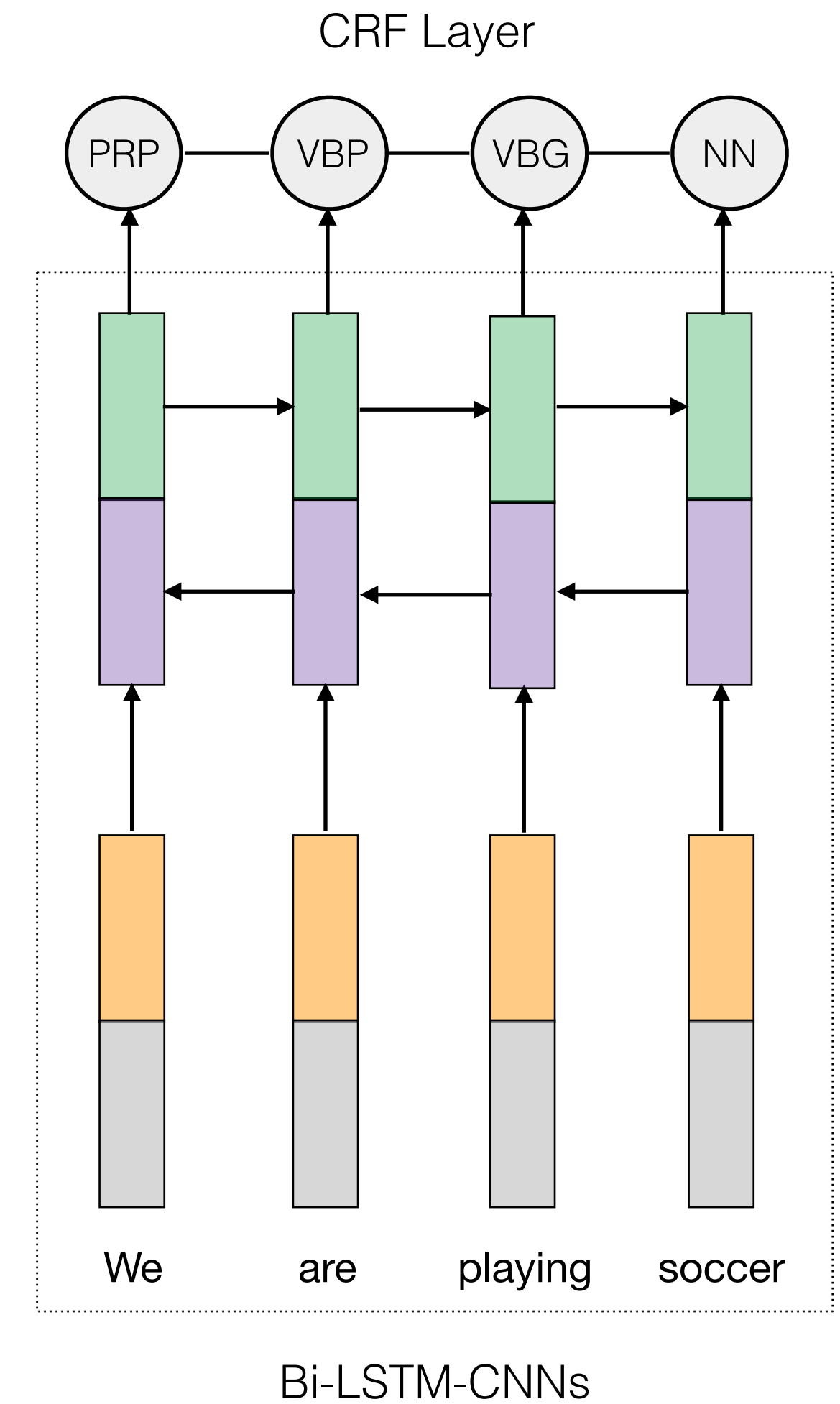
CRF has weaker independence assumption than MEMMs!

Sequence Labeling: BLSTM-CNNs-CRF

$$v \cdot f(x_1, \dots, x_m, j, s_{j-1}, s_j) = \mathbf{W}_{s_{j-1}, s_j}^T \mathbf{h}_j$$

Parameters w.r.t s_{j-1}, s_j
LSTM hidden state at j

$$p(s \mid \mathbf{x}; \mathbf{W}) = \frac{\prod_{j=1}^n \exp \left(\mathbf{w}_{s_{j-1}, y_j} \mathbf{h}_j \right)}{\sum_{s' \in \mathcal{S}} \prod_{j=1}^n \exp \left(\mathbf{w}_{s'_{j-1}, s'_j} \mathbf{h}_j \right)}$$



Sequence Labeling

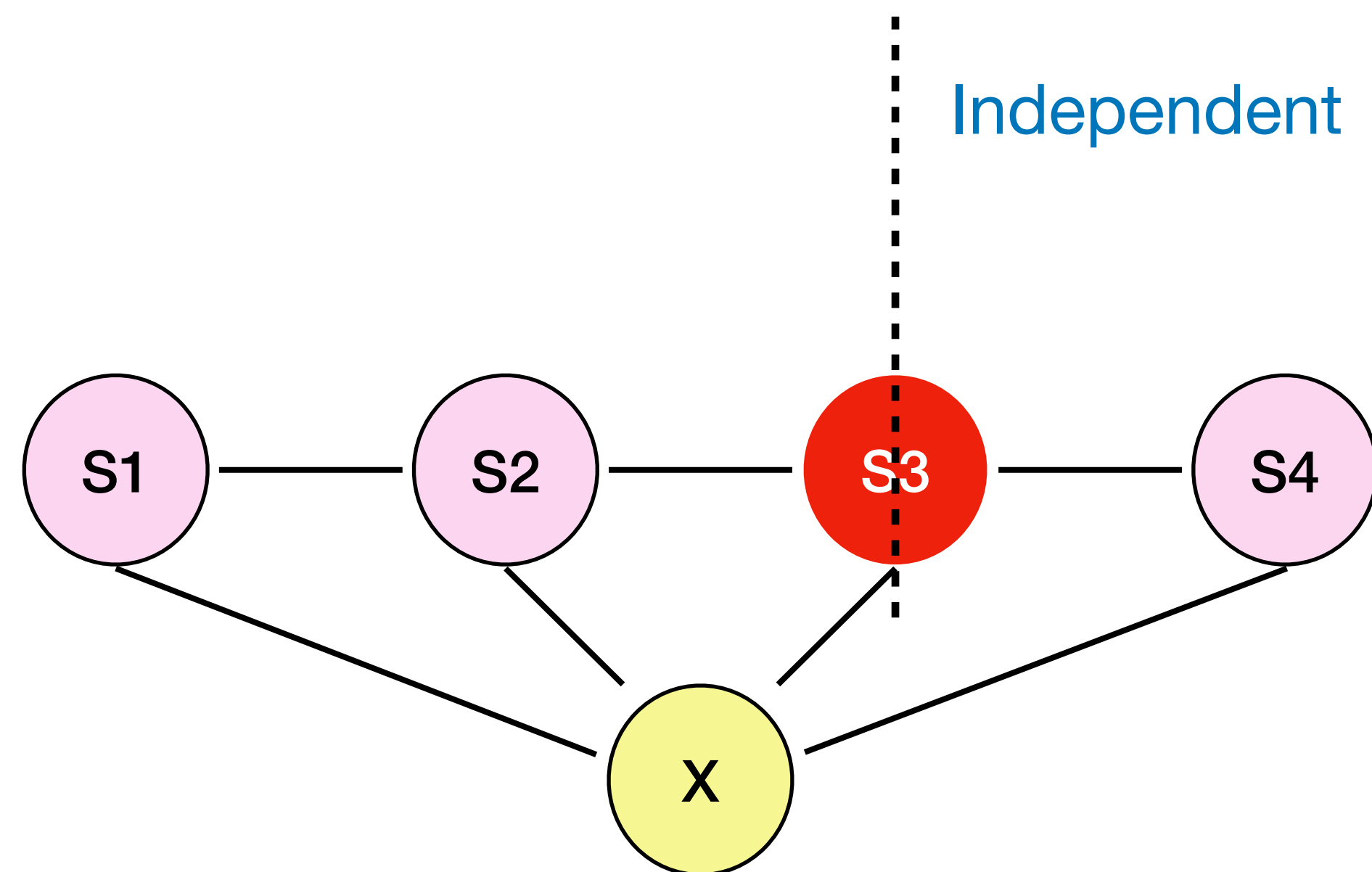
- **BLSTM-CNN-CRF**

	POS Tagging	NER
Classic SoTA	97.3%	91.2%
BLSTM	96.9%	87.0%
BLSTM-CNN	97.3%	89.4%
BLSTM-CNN-CRF	97.6%	91.2%

Structured models are still useful!

How to Compute CRFs?

$$p(s_1, \dots, s_m | x_1, \dots, x_m) = \frac{\prod_{j=1}^m \exp(v \cdot f(x_1, \dots, x_m, i, s_{j-1}, s_j))}{\sum_{s'_1, \dots, s'_m \in \mathcal{S}} \prod_{j=1}^m \exp(v \cdot f(x_1, \dots, x_m, i, s'_{j-1}, s'_j))}$$



CRF

Viterbi Algorithm

- ▶ Goal: for a given input sequence $x_1, \dots, x_m$, find

$$\arg \max_{s_1, \dots, s_m} p(s_1 \dots s_m | x_1 \dots x_m)$$

- ▶ We can use the *Viterbi* algorithm again (see last lecture on HMMs). Basic data structure:

$$\pi[j, s]$$

will be a table entry that stores the maximum probability for any state sequence ending in state s at position j . More formally:

$$\pi[j, s] = \max_{s_1 \dots s_{j-1}} \left(p(s | s_{j-1}, x_1 \dots x_m) \prod_{k=1}^{j-1} p(s_k | s_{k-1}, x_1 \dots x_m) \right)$$

Viterbi Algorithm

- Initialization: for $s \in \mathcal{S}$

$$\pi[1, s] = p(s|s_0, x_1 \dots x_m)$$

where s_0 is a special “initial” state.

- For $j = 2 \dots m$, $s = 1 \dots k$:

$$\pi[j, s] = \max_{s' \in \mathcal{S}} [\pi[j - 1, s'] \times p(s|s', x_1 \dots x_m)]$$

- We then have

$$\max_{s_1 \dots s_m} p(s_1 \dots s_m | x_1 \dots x_m) = \max_s \pi[m, s]$$

Reading Materials

- **Relavant Papers**
 - BLSTM-CNNs-CRF